



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas para Toma de Decisión Artificial en Escenarios Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos limitados
bajo incertidumbre utilizando como escenario de prueba una crisis
pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis compara distintos métodos de aprendizaje por refuerzo en un problema de asignación de recursos limitados bajo incertidumbre, con una pandemia simulada como banco de pruebas. En cada paso llega una ciudad con un nivel de severidad y el agente decide cuántos recursos le asigna de un presupuesto fijo. La severidad que no se atiende crece en los pasos siguientes, de modo que una asignación insuficiente al principio se paga más adelante. El trabajo se implementó en mPES, una extensión del entorno de pandemia de la Universidad de Essex.

Se evaluaron seis modelos individuales, con hiperparámetros ajustados por optimización bayesiana. Dos son métodos tabulares (Q-Learning y Double Q-Learning) y cuatro son redes neuronales: una red profunda (DQN), una variante recurrente que recuerda los últimos estados (DQN recurrente, con una capa LSTM), un método actor-crítico (A2C) y un Transformer causal, que atiende a la historia reciente de la secuencia. También se evaluaron seis ensambles que combinan las decisiones de varias de esas redes. Todos se probaron sobre las mismas 64 secuencias, primero en las condiciones de entrenamiento y después en 21 escenarios de generalización que cambian la distribución de severidades, la longitud de las secuencias o ambas. Tres de ellos llevan la severidad o la longitud por encima de las cotas del entorno.

El desempeño se mide como la fracción de la mejora posible que logra el agente en cada secuencia: 0 equivale a no asignar ningún recurso y 1 a la asignación óptima, calculada conociendo de antemano la secuencia completa. Un agente que decide al azar obtiene 0,670.

El Transformer fue el mejor modelo individual: obtuvo 0,927 en las condiciones de entrenamiento, frente a 0,887 de Q-Learning, es decir, redujo en alrededor de un 36 % la distancia al óptimo. En los escenarios de generalización mantuvo ese nivel (0,930, frente a 0,871 de Q-Learning). El mejor sistema fue un ensamble que combina DQN, la red recurrente y el Transformer y agrega dos reglas fijas (un *prior* de severidad y una cota de seguridad), con 0,939 en los escenarios de generalización. Los peores resultados fueron los de los métodos tabulares cuando la severidad supera la cota del entorno (0,76 y 0,79, frente a 0,996 del Transformer). Para la mayoría de las redes profundas, el caso más difícil fueron las secuencias más largas que las de entrenamiento.

Los ensambles ponderan a cada modelo según su confianza en cada decisión, medida como uno menos la entropía normalizada de su distribución de acciones. El único que superó al Transformer le da a éste la mayor parte del peso, pero en la referencia su decisión final difiere de la del Transformer en el 61 % de los casos, principalmente por el *prior* de severidad; los que sólo promedian o votan quedaron por debajo.

Palabras clave: aprendizaje por refuerzo, asignación secuencial de recursos, Transformer causal, ensambles de modelos, entropía de Shannon, generalización.

Abstract

This thesis compares several reinforcement learning methods on a problem of allocating limited resources under uncertainty, using a simulated pandemic as a test bed. At each step a city arrives with a given severity and the agent decides how many resources to assign to it from a fixed budget. Severity that is not addressed grows over the following steps, so an insufficient allocation early on is paid for later. The work was implemented in mPES, an extension of the University of Essex pandemic environment.

Six individual models were evaluated, with hyperparameters tuned by Bayesian optimisation. Two are tabular methods (Q-Learning and Double Q-Learning) and four are neural networks: a deep network (DQN), a recurrent variant that remembers the most recent states (recurrent DQN, with an LSTM layer), an actor-critic method (A2C) and a causal Transformer, which attends to the recent history of the sequence. Six ensembles that combine the decisions of several of those networks were also evaluated. All models were tested on the same 64 sequences, first under the training conditions and then in 21 generalisation scenarios that change the distribution of severities, the length of the sequences, or both. Three of them push severity or length above the bounds of the environment.

Performance is measured as the fraction of the achievable improvement that the agent attains in each sequence: 0 means assigning no resources at all and 1 means the optimal allocation, computed with full knowledge of the sequence in advance. As a reference, an agent that decides at random scores 0.670.

The Transformer was the best individual model: it scored 0.927 under the training conditions, against 0.887 for Q-Learning, which amounts to reducing the distance to the optimum by about 36%. It kept that level in the generalisation scenarios (0.930, against 0.871 for Q-Learning). The best system overall was an ensemble that combines DQN, the recurrent network and the Transformer and adds two fixed rules (a severity prior and a safety bound), with 0.939 in the generalisation scenarios. The worst results came from the tabular methods when severity exceeded the bound of the environment (0.76 and 0.79, against 0.996 for the Transformer). For most deep networks, the hardest case was sequences longer than those seen in training.

The ensembles weight each model by its confidence in each decision, measured as one minus the normalised entropy of its action distribution. The only one that outperformed the Transformer gives it most of the weight, but under the training conditions its final decision differs from the Transformer's in 61% of the cases, mainly because of the severity prior; those that only average or vote fell below it.

Keywords: reinforcement learning, sequential resource allocation, causal Transformer, model ensembles, Shannon entropy, generalisation.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	1
1.3. Preguntas e hipótesis de trabajo	1
1.4. Contribuciones	2
1.5. Estructura del documento	2
2. Marco Teórico	3
2.1. Procesos de decisión de Markov	3
2.2. Q-Learning tabular	3
2.2.1. Double Q-Learning	4
2.2.2. Modelado de recompensa mediante potencial (PBRS)	4
2.3. Deep Q-Networks (DQN)	4
2.3.1. Double DQN	5
2.4. Redes recurrentes y LSTM	5
2.5. Gradiente de política y actor-crítico	5
2.6. Atención y Transformer	6
2.7. Entropía de Shannon como medida de confianza	6
2.8. Ensamblados de modelos	7
2.9. Optimización bayesiana de hiperparámetros	8
2.10. Evaluación de la generalización	8
2.11. Comparación estadística entre modelos	8
3. Estado de la Cuestión	10
3.1. Aprendizaje por refuerzo profundo	10
3.2. Modelado secuencial con Transformers	10
3.3. Ensamblados de modelos	11
3.4. Generalización bajo variaciones controladas del entorno	11
4. Materiales y Métodos	12
4.1. Implementación	12

4.2. Entorno PES	12
4.2.1. Estados y acciones	12
4.2.2. Dinámica y recompensa	13
4.2.3. Secuencias de evaluación	13
4.3. Métrica de desempeño	14
4.4. Modelos individuales	14
4.4.1. Modelos tabulares	15
4.4.2. Modelos profundos	17
4.5. Ensamblés	18
4.5.1. Notación común	19
4.5.2. Ensamble ponderado	21
4.5.3. Voto suave	22
4.5.4. Voto por acción	22
4.5.5. Consenso	22
4.5.6. Consenso con <i>prior</i>	23
4.5.7. Compuerta del Transformer	23
4.6. Optimización de hiperparámetros	24
4.7. Escenarios de generalización	24
4.8. Protocolo de evaluación	25
5. Resultados	27
5.1. Mejor modelo individual	27
5.1.1. Desempeño por familia de escenarios	28
5.1.2. Tamaño de la ventaja del Transformer	29
5.1.3. Resultados por modelo en la referencia	29
5.1.4. Curvas por escenario	33
5.2. Ensamblés	34
5.2.1. Cuánto cambian las reglas fijas la decisión	35
6. Discusión	38
6.1. Interpretación de la ventaja del Transformer	38
6.2. Efecto de la regla de combinación	39
7. Conclusiones	41

7.1. Respuesta a las preguntas	41
7.2. Limitaciones del estudio	41
7.3. Trabajo futuro	42
7.4. Conclusión general	43
Referencias Bibliográficas	44
Apéndice	47
A. Comandos de reproducción	47
B. Ejecución de la evaluación	47
C. Mapas estadísticos complementarios	48
D. Confianza registrada del Transformer	51

Índice de figuras

4.1. Agente aleatorio	15
5.1. Desempeño medio por escenario (modelos individuales)	28
5.2. Resultados por secuencia: Q-Learning base	30
5.3. Resultados por secuencia: Q-Learning	30
5.4. Resultados por secuencia: Double Q-Learning	31
5.5. Resultados por secuencia: DQN	31
5.6. Resultados por secuencia: DQN recurrente	32
5.7. Resultados por secuencia: A2C	32
5.8. Resultados por secuencia: Transformer	33
5.9. Curvas por secuencia en seis escenarios (modelos individuales)	34
5.10. Desempeño medio por escenario (ensambles)	36
5.11. Curvas de los ensambles en los escenarios fuera de rango	36
5.12. Resultados por secuencia: ensamble ponderado	37
C.1. Significancia de Welch por escenario (modelos individuales)	48
C.2. Divergencia KL de acciones por escenario (modelos individuales)	49
C.3. d de Cohen entre pares de modelos individuales	49
C.4. Comparación estadística por escenario (ensambles)	50
C.5. Comparación entre pares de ensambles	50
D.1. Confianza del Transformer en cada decisión	51

Índice de tablas

4.1. Modelos individuales	15
4.2. Hiperparámetros de los modelos tabulares	16
4.3. Hiperparámetros de los modelos de tipo DQN	19
4.4. Hiperparámetros de A2C	19
4.5. Ensamblés evaluados	20
4.6. Parámetros de los ensambles	24
4.7. Escenarios de evaluación	26
5.1. Desempeño de los modelos individuales	27
5.2. Comparación del Transformer con otros modelos	29
5.3. Desempeño de los ensambles	34
5.4. Frecuencia con que las reglas cambian la decisión	35
5.5. Comparación del ensamble ponderado con otros ensambles	37
A.1. Comandos de reproducción	47

1. Introducción

Esta tesis compara arquitecturas de aprendizaje por refuerzo (RL, *Reinforcement Learning*) en un problema de asignación secuencial de recursos bajo incertidumbre. Un agente reparte un presupuesto fijo entre ciudades que llegan de a una, sin saber cuántas faltan ni cuán graves serán, y cada unidad que asigna deja de estar disponible para las siguientes. El entorno de experimentación, **mPES**, extiende el *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1], que usa una pandemia simulada como contexto.

1.1. Motivación

La pandemia de COVID-19 mostró este tipo de problema a gran escala: gobiernos y hospitales debieron repartir recursos escasos sabiendo que cada asignación modificaba la evolución posterior del sistema sanitario. En este trabajo la pandemia no es el objeto de estudio, sino un escenario controlado y reproducible. En él se puede medir qué modelos deciden mejor y cómo se comportan fuera de las condiciones de entrenamiento.

1.2. Definición del problema

Un agente de RL debe distribuir un *presupuesto limitado* de recursos entre ciudades que se incorporan de a una. En cada paso decide cuántos recursos, entre 0 y 10, asignar a la ciudad que acaba de llegar. La severidad de cada ciudad crece un 40% en cada paso, y cada unidad de recurso asignada la reduce en 0,4 (Ecuación (4.2)). Esta regla es una simplificación deliberada de los modelos compartimentales de la epidemiología computacional [2], como las simulaciones espacialmente resueltas de la propagación de COVID-19 en tres países europeos de Parisi *et al.* [3]: conserva sólo el crecimiento de la severidad y el efecto de los recursos asignados. El Capítulo 4 detalla la formulación como MDP (*Markov Decision Process*) y las 64 secuencias de evaluación.

1.3. Preguntas e hipótesis de trabajo

El trabajo responde dos preguntas:

1. ¿Qué arquitectura individual toma mejores decisiones, y cuál conserva mejor su desempeño cuando cambian la severidad o la longitud de las secuencias?
2. ¿Combinar varios modelos ya entrenados en un ensamble mejora al mejor modelo individual? Si es así, ¿con qué regla de combinación?

Para la primera pregunta se plantea la siguiente hipótesis:

H1. *Entre los modelos individuales evaluados, el Transformer causal alcanza el mayor desempeño medio y lo conserva en los escenarios de generalización.*

La segunda se responde de forma exploratoria, comparando seis reglas de combinación.

1.4. Contribuciones

1. Una comparación de seis implementaciones de RL bajo el mismo entorno, el mismo protocolo y las mismas 64 secuencias de evaluación. Son dos tabulares (Q-Learning y Double Q-Learning) y cuatro profundas (DQN, DQN recurrente, A2C y Transformer causal), todas con hiperparámetros ajustados por optimización bayesiana.
2. Seis reglas para combinar esos modelos en un ensamble sin reentrenarlos; cada modelo pesa según su confianza, medida con la entropía de su distribución de acciones.
3. Una evaluación en 21 escenarios de generalización que cambian la severidad inicial, la longitud de las secuencias o ambas; tres de ellos superan las cotas de severidad o de longitud del entorno.

1.5. Estructura del documento

Los Capítulos 2 y 3 presentan el marco teórico y los trabajos relacionados. El Capítulo 4 describe el entorno, los modelos, los ensambles y el protocolo de evaluación; el Capítulo 5, los resultados; el Capítulo 6, su discusión, y el Capítulo 7, las conclusiones. El Apéndice reúne los comandos de reproducción, los mapas estadísticos de apoyo y el registro de confianza del Transformer.

2. Marco Teórico

Este capítulo resume los conceptos que se usan en el resto de la tesis. Cubre los procesos de decisión de Markov, los algoritmos de aprendizaje por refuerzo implementados, la entropía como medida de confianza, los ensambles de modelos, la optimización bayesiana de hiperparámetros y las pruebas estadísticas con que se comparan los resultados. La referencia general es el texto de Sutton y Barto [4].

2.1. Procesos de decisión de Markov

Un MDP finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde $\mathcal{S}$ es un espacio de estados finito, $\mathcal{A}$ un espacio de acciones finito, $P(s' \mid s, a)$ la probabilidad de transición, $R(s, a, s')$ una función de recompensa acotada y $\gamma \in [0, 1)$ el factor de descuento. Bajo una política $\pi(a \mid s)$ el agente genera una trayectoria $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ cuyo *retorno descontado* desde el tiempo t es:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.1)$$

Donde r_{t+k+1} es la recompensa recibida $k + 1$ pasos después de t . La función de valor de estado-acción de la política es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ y la política óptima satisface la ecuación de Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (2.2)$$

Donde la esperanza se toma sobre el estado siguiente $s' \sim P(\cdot \mid s, a)$ y la recompensa $r = R(s, a, s')$.

2.2. Q-Learning tabular

Cuando $\mathcal{S}$ y $\mathcal{A}$ son enumerables, Q se almacena como una tabla y se aplica la actualización de Watkins [5]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.3)$$

Donde $\alpha \in (0, 1]$ es la tasa de aprendizaje. Las acciones se eligen con una política ε -greedy: con probabilidad ε se elige una acción al azar y, en caso contrario, $\arg \max_a Q(s, a)$.

2.2.1. Double Q-Learning

El operador $\max$ de la Ecuación (2.3) sobreestima el valor de las acciones. *Double Q-Learning* [6] mantiene dos tablas Q^A y Q^B y usa una para elegir la acción siguiente y la otra para evaluarla:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', \arg \max_{a'} Q^A(s', a')) - Q^A(s, a)] \quad (2.4)$$

La actualización de Q^B es simétrica, y en cada paso se elige con probabilidad 1/2 cuál de las dos tablas se actualiza.

2.2.2. Modelado de recompensa mediante potencial (PBRs)

El *Potential-Based Reward Shaping* (PBRs) [7] suma a la recompensa el término $F(s, s') = \gamma\Phi(s') - \Phi(s)$, donde $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ es una función de potencial sobre los estados. Ng *et al.* demostraron que este término no cambia las políticas óptimas del MDP original. En Double Q-Learning, el potencial es la suma de las severidades actuales de las ciudades con signo negativo, $\Phi(s) = -\sum_i S_i$, y el término se multiplica por un coeficiente κ : la recompensa efectiva es $\tilde{r} = r + \kappa F(s, s')$.

2.3. Deep Q-Networks (DQN)

Cuando el espacio de estados es grande, $Q(s, a; \theta)$ se aproxima con una red neuronal de parámetros θ . El *Deep Q-Network* [8] estabiliza el entrenamiento con dos elementos. El primero es un *buffer* de repetición de experiencia (*experience replay*) [9], del que se muestrean lotes de transiciones al azar para romper la correlación entre muestras consecutivas. El segundo es una *red objetivo* (*target network*) $Q(\cdot; \theta^-)$, cuyos parámetros se copian desde θ cada C pasos. La función de costo es:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.5)$$

Donde $\mathcal{D}$ es el *buffer* de transiciones almacenadas.

2.3.1. Double DQN

Double DQN [10] aplica la idea de la Ecuación (2.4) sin agregar una segunda red: la acción siguiente se elige con la red en línea θ y se evalúa con la red objetivo θ^- :

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) \quad (2.6)$$

Los tres agentes de tipo DQN de mPES (DQN, DQN recurrente y Transformer) usan este blanco, enmascaran las acciones no factibles antes del $\arg \max$ y reemplazan el error cuadrático por la pérdida de Huber [11].

2.4. Redes recurrentes y LSTM

Cuando la información relevante para decidir no está toda en el estado actual, es habitual procesar una secuencia de estados con una red recurrente. La *Long Short-Term Memory* (LSTM) [12] usa compuertas de entrada i_t , olvido f_t y salida o_t , con valores en $(0, 1)$, que controlan cómo el estado de celda c_t acumula información:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad h_t = o_t \odot \tanh(c_t) \quad (2.7)$$

Donde $\tilde{c}_t$ es el candidato de celda, h_t el estado oculto y $\odot$ el producto elemento a elemento. Hausknecht y Stone [13] mostraron que incorporar una capa LSTM en un DQN (*Deep Recurrent Q-Network*) mejora el desempeño en entornos parcialmente observables.

2.5. Gradiente de política y actor–crítico

Los métodos de *gradiente de política* [14] parametrizan directamente la política $\pi_\theta(a \mid s)$ y ajustan θ en la dirección que aumenta el retorno esperado $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s) A^{\pi_{\theta}}(s, a)] \quad (2.8)$$

Donde $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ es la *ventaja* y $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$. El *Advantage Actor-Critic* (A2C) [15] entrena a la vez un actor, que produce π_{θ} , y un crítico, que estima V_{ϕ} . La ventaja se estima con *Generalized Advantage Estimation* (GAE) [16]:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k} \quad \delta_t = r_{t+1} + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t) \quad (2.9)$$

Donde δ_t es el error de diferencia temporal de un paso y $\lambda \in [0, 1]$ regula el compromiso entre sesgo y varianza. El actor se regulariza con un término $-\beta_H H(\pi_{\theta}(\cdot | s))$, con $\beta_H \geq 0$, que evita que la política se concentre demasiado pronto en una acción.

2.6. Atención y Transformer

El *Transformer* [17] reemplaza la recurrencia por *atención de producto escalar escalado*. Dadas las matrices de consultas Q , claves K y valores V , obtenidas por proyecciones lineales de la secuencia de entrada, y siendo d_k la dimensión de las claves, cada posición combina los valores de las demás con pesos $\text{softmax}(QK^{\top} / \sqrt{d_k})$. Para que cada posición sólo use información pasada se suma una máscara triangular M , con $M_{ij} = -\infty$ para $j > i$ y $M_{ij} = 0$ en otro caso:

$$\text{Atención}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right) V \quad (2.10)$$

2.7. Entropía de Shannon como medida de confianza

Dada una distribución de probabilidad discreta $p(a | s)$ sobre las $|\mathcal{A}|$ acciones, su *entropía de Shannon* [18] es:

$$H(p) = - \sum_{a \in \mathcal{A}} p(a | s) \log p(a | s) \quad (2.11)$$

Con la convención $0 \log 0 = 0$. Su valor máximo, $\log |\mathcal{A}|$, corresponde a la distribución

uniforme, de modo que la versión normalizada

$$H_{\text{norm}}(p) = \frac{H(p)}{\log |\mathcal{A}|} \in [0, 1] \quad (2.12)$$

vale 0 cuando toda la probabilidad está en una acción y 1 cuando todas las acciones son igualmente probables. La entropía de la distribución de salida se usa para cuantificar la incertidumbre de un clasificador; por ejemplo, Lakshminarayanan *et al.* [19] la emplean para detectar entradas ajenas a la distribución de entrenamiento. En esta tesis se usa $1 - H_{\text{norm}}$ como *confianza* del agente en cada decisión. Cuando el modelo produce valores Q y no probabilidades, primero se convierten en una distribución (Sección 4.5), por lo que la confianza resultante es una medida heurística y no una probabilidad calibrada.

2.8. Ensambls de modelos

Un *ensemble* combina las salidas de K modelos ya entrenados para producir una única decisión. Sea $p_k(a \mid s)$ la distribución sobre acciones del miembro k . Si el miembro produce valores $Q_k(s, \cdot)$, la distribución se obtiene con una *softmax* de temperatura τ :

$$p_k(a \mid s) = \frac{e^{Q_k(s,a)/\tau}}{\sum_{a'} e^{Q_k(s,a')/\tau}} \quad (2.13)$$

Un τ pequeño concentra la distribución en la acción de mayor valor y uno grande la acerca a la uniforme. La regla de combinación más habitual es el *voto suave* (*soft voting*), un promedio ponderado de las distribuciones:

$$p_{\text{ens}}(a \mid s) = \frac{\sum_{k=1}^K w_k p_k(a \mid s)}{\sum_{k=1}^K w_k} \quad w_k \geq 0 \quad (2.14)$$

Donde w_k es el peso del miembro k ; la acción elegida es $\arg \max_a p_{\text{ens}}(a \mid s)$. En el *voto duro* (*hard voting*), en cambio, cada miembro vota sólo por su acción preferida $a_k = \arg \max_a p_k(a \mid s)$ y gana la más votada:

$$a_{\text{ens}} = \arg \max_a \sum_{k=1}^K w_k \mathbf{1}[a_k = a] \quad (2.15)$$

Donde $\mathbf{1}[\cdot]$ vale 1 si la condición se cumple y 0 si no. Los pesos pueden ser fijos o depender de la confianza de cada miembro en la decisión actual, $c_k = 1 - H_{\text{norm}}(p_k)$ (Ecuación (2.12)): así, un miembro con una distribución más concentrada pesa más.

2.9. Optimización bayesiana de hiperparámetros

Los agentes de RL tienen hiperparámetros no diferenciables (α , γ , tamaño del *buffer*, decaimiento de ε , anchura de la red, entre otros) que interactúan entre sí. La *optimización bayesiana* trata el puntaje de validación $y = f(x)$ de una configuración x como una función de caja negra y la explora con un modelo probabilístico auxiliar. En esta tesis se usa el estimador estructurado en árbol de Parzen (TPE) [20] de Optuna [21]. Dado un umbral y^* , fijado como un cuantil de los puntajes observados, TPE estima por separado las densidades $\ell(x) = p(x \mid y < y^*)$ y $g(x) = p(x \mid y \geq y^*)$ con estimadores de densidad por núcleos, que suavizan las configuraciones observadas. Como el objetivo es maximizar el puntaje, TPE propone nuevas configuraciones donde el cociente $g(x)/\ell(x)$ es alto, es decir, donde las configuraciones buenas son más frecuentes que las malas.

2.10. Evaluación de la generalización

Sea P_{train} la distribución de las secuencias usadas en el entrenamiento y P_{test} la de las secuencias con que se evalúa la política. En la condición de referencia ambas coinciden, $P_{\text{test}} = P_{\text{train}}$. En esta tesis se llama *escenario de generalización* a una condición de evaluación en la que $P_{\text{test}} \neq P_{\text{train}}$. En esos escenarios cambia la distribución de las severidades iniciales, la de las longitudes de secuencia o ambas; la regla de transición (Ecuación (4.2)) y la recompensa no cambian. Los nuevos valores pueden quedar dentro del rango visto en el entrenamiento (interpolación) o fuera de él (extrapolación) [22].

2.11. Comparación estadística entre modelos

Para comparar dos muestras de desempeño A y B , con n_A y n_B observaciones (en mPES, el desempeño de cada secuencia), se usan tres medidas:

- **Tamaño del efecto:** la d de Cohen [23] expresa la diferencia de medias en unidades

de desviación estándar:

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p} \quad s_p = \sqrt{\frac{(n_A - 1) s_A^2 + (n_B - 1) s_B^2}{n_A + n_B - 2}} \quad (2.16)$$

Donde $\bar{x}$ y s^2 son la media y la varianza muestral de cada grupo y s_p la desviación estándar combinada. Valores de $|d|$ cercanos a 0,2, 0,5 y 0,8 se suelen interpretar como efectos pequeño, medio y grande.

- **Significancia:** la prueba t de Welch [24] contrasta la igualdad de medias sin suponer varianzas iguales:

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_A^2/n_A + s_B^2/n_B}} \quad (2.17)$$

Los grados de libertad se estiman con la aproximación de Welch–Satterthwaite. El valor p bilateral es la probabilidad de observar una diferencia al menos tan grande si las medias fueran iguales; un p pequeño indica que la diferencia no se explica por azar, pero no dice cuán grande es. En las tablas y figuras se reporta $\log_{10} p$.

- **Diferencia entre distribuciones:** la divergencia de Kullback–Leibler entre dos distribuciones discretas p y q es:

$$D_{\text{KL}}(p \parallel q) = \sum_a p(a) \log \frac{p(a)}{q(a)} \quad (2.18)$$

Vale 0 cuando ambas coinciden y crece con la discrepancia. Se usa para comparar la distribución de acciones de un modelo en un escenario con la de la referencia, y, en forma simetrizada $\frac{1}{2}[D_{\text{KL}}(p \parallel q) + D_{\text{KL}}(q \parallel p)]$, para comparar los histogramas de desempeño de dos modelos. Para evitar $\log 0$ se suma $\varepsilon = 10^{-9}$ y se renormaliza.

3. Estado de la Cuestión

Este trabajo se apoya en tres líneas: el aprendizaje por refuerzo profundo, los modelos secuenciales basados en atención y la evaluación de la generalización en RL. El punto de partida es el entorno *Pandemic Experiment Scenario* (PES) [1], desarrollado por el BCI-NE Lab de la Universidad de Essex para estudiar la decisión conjunta en equipos humano-máquina [25]. Los datos de un experimento en el que nueve participantes decidían junto a un agente artificial están publicados como el conjunto `ds004477` [26]. Esta tesis sólo toma el entorno: no trabaja con participantes ni con esos datos, sino con agentes artificiales.

3.1. Aprendizaje por refuerzo profundo

El RL pasó de métodos tabulares como Q-Learning [5] a aproximaciones capaces de manejar espacios de estados grandes [4]. Los trabajos en los que se apoyan los modelos de esta tesis son la red Q profunda de Mnih *et al.* [8], con repetición de experiencia y red objetivo; las correcciones de la sobreestimación de Double Q-Learning [6] y Double DQN [10]; y los métodos actor-crítico [15]. Sus fundamentos se resumen en el Capítulo 2.

3.2. Modelado secuencial con Transformers

La arquitectura Transformer [17] usa la atención causal (Ecuación (2.10)) para combinar todas las posiciones previas de una ventana de estados. En RL se la usó de dos maneras. El *Decision Transformer* [27] trata el RL como modelado de secuencias: aprende de trayectorias ya registradas y genera acciones condicionadas al retorno deseado. La otra manera la usa como codificador de la historia en un agente que aprende interactuando con el entorno. En ese caso, Parisotto *et al.* [28] mostraron que el Transformer estándar es difícil de entrenar y que, con la normalización de capa aplicada antes de cada subcapa y compuertas en las conexiones residuales, iguala o supera a las LSTM en tareas que requieren memoria. El Transformer de esta tesis sigue la segunda manera: es un DQN cuyo codificador procesa los últimos estados.

En mPES el estado no incluye la severidad de las ciudades ya atendidas, que suma a

la recompensa, ni la longitud de la secuencia (Sección 4.2.2). Un modelo que ve los estados recientes dispone de información para estimar mejor el retorno de cada acción. Si eso se traduce en mejores decisiones es una pregunta empírica, que esta tesis responde comparando modelos con y sin ventana.

3.3. Ensamblados de modelos

Combinar varios modelos es una forma habitual de obtener predicciones más estables. Lakshminarayanan *et al.* [19] mostraron que promediar redes de la misma arquitectura, entrenadas desde inicializaciones distintas, mejora la estimación de la incertidumbre. En RL, Wiering y van Hasselt [29] combinaron en un mismo agente las políticas de cinco algoritmos distintos, entre ellos Q-learning, Sarsa y actor-crítico, mediante votación y combinaciones de distribuciones de Boltzmann. En sus laberintos, el voto mayoritario y la multiplicación de Boltzmann superaron a los algoritmos individuales. Los ensambles de esta tesis siguen esa segunda línea, con dos diferencias: combinan modelos ya entrenados, sin reentrenarlos, y ponderan a cada uno por la entropía de su distribución de acciones.

3.4. Generalización bajo variaciones controladas del entorno

Varios trabajos mostraron que evaluar un agente en las mismas condiciones del entrenamiento sobreestima la calidad de lo aprendido. Henderson *et al.* [30] encontraron que los algoritmos habituales son sensibles a la semilla, a los hiperparámetros y a detalles de implementación; Cobbe *et al.* [31] mostraron que agentes con buen rendimiento en los niveles de entrenamiento de ProcGen fallaban en niveles nuevos generados con la misma regla; Packer *et al.* [22] distinguieron entre interpolación y extrapolación de los parámetros del entorno; y Kirk *et al.* [32] recomiendan evaluar sobre familias de variaciones controladas del entorno.

mPES sigue esa recomendación: además de la condición de referencia, evalúa cada modelo en 21 escenarios de generalización, tres de ellos por encima de las cotas del entorno (Sección 4.7).

4. Materiales y Métodos

Este capítulo describe el entorno, la métrica de desempeño, la implementación de los modelos individuales y de los ensambles, la optimización de hiperparámetros y el protocolo de evaluación.

4.1. Implementación

Los experimentos se implementaron en el repositorio **mPES** [33], que extiende el repositorio **PES** del BCI-NE Lab [1], del que hereda el entorno y la implementación tabular de Q-Learning. El código está organizado en tres grupos de paquetes: **tabular/** (métodos tabulares), **ml/** (redes neuronales) y **ens/** (ensambles). Cada paquete incluye el entorno, implementado con Gymnasium [34], su rutina de entrenamiento y, cuando corresponde, su estudio de optimización bayesiana. Se usó Python 3.12 con TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17. Los comandos para reproducir cada resultado están en el Apéndice A.

4.2. Entorno PES

4.2.1. Estados y acciones

El entorno se implementa como un MDP finito. En cada paso (*trial*) llega una ciudad y el agente decide cuántas unidades de un recurso escaso le asigna. El estado que observa el agente es:

$$\mathcal{S} = \{(R, t, S) : R \in \{0, \dots, 30\}, t \in \{0, \dots, T_{\text{máx}}\}, S \in \{0, \dots, S_{\text{máx}}\}\} \quad (4.1)$$

Donde R son los recursos que quedan, t el número de paso dentro de la secuencia ($T_{\text{máx}} = 10$) y S la severidad inicial de la ciudad actual ($S_{\text{máx}} = 9$); hay $31 \times 11 \times 10 = 3410$ estados. Cada secuencia dispone de 39 recursos, de los cuales el entorno reserva 9 como preasignados, por lo que el agente administra $R \leq 30$. Las acciones son $\mathcal{A} = \{0, 1, \dots, 10\}$; una asignación mayor que R se recorta a los recursos disponibles.

4.2.2. Dinámica y recompensa

En cada paso, la severidad de toda ciudad i ya incorporada se actualiza con:

$$S_{i,t+1} = \max(0, \beta S_{i,t} - \alpha a_i) \quad \alpha = 0,4 \quad \beta = 1 + \alpha = 1,4 \quad (4.2)$$

Donde a_i es la asignación que recibió la ciudad i al incorporarse, α la eficacia de cada unidad de recurso y β el factor de crecimiento por paso. La asignación de una ciudad se fija una sola vez, pero su severidad se actualiza en todos los pasos siguientes; por eso una asignación insuficiente se amplifica geométricamente. La recompensa de cada paso es la suma, con signo negativo, de las severidades de las ciudades activas:

$$r_t = - \sum_{i \leq t} S_{i,t+1} \quad (4.3)$$

Maximizar el retorno equivale, por lo tanto, a minimizar la severidad acumulada de la secuencia.

El estado que observa el agente no incluye la severidad de las ciudades ya incorporadas ni la longitud de la secuencia. Esa severidad forma parte de la recompensa, pero no depende de las decisiones posteriores, porque la asignación de cada ciudad se fija al llegar. Por lo tanto no cambia qué acción conviene en cada paso, aunque sí agrega al retorno una parte que el estado actual no explica. En sentido estricto, (R, t, S) es una observación parcial del entorno.

4.2.3. Secuencias de evaluación

Cada experimento se organiza en 8 bloques de 8 secuencias, de 3 a 10 pasos cada una, lo que suma 360 pasos. La evaluación de referencia usa las $n = 64$ secuencias fijas de los archivos `initial_severity.csv` y `sequence_lengths.csv`, idénticos en todos los paquetes, de modo que todos los modelos enfrentan exactamente las mismas secuencias. En esos archivos las severidades iniciales van de 2 a 8.

El entrenamiento no recorre esas 64 secuencias en orden: en cada episodio sortea una longitud y unas severidades iniciales a partir de sus distribuciones empíricas en los mismos

archivos. Por eso, durante el entrenamiento las severidades sólo toman valores de 2 a 8, aunque el espacio de estados admite de 0 a 9.

4.3. Métrica de desempeño

La severidad final acumulada de una secuencia de T pasos es:

$$S_{\text{cruda}} = \sum_{i=1}^T S_i^{\text{final}} \quad (4.4)$$

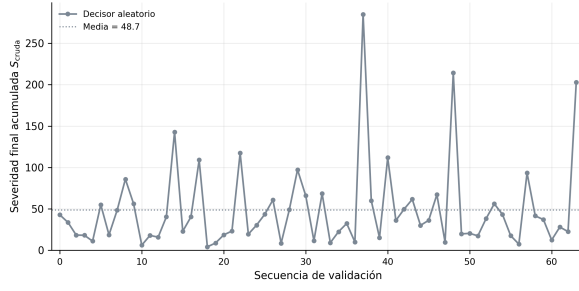
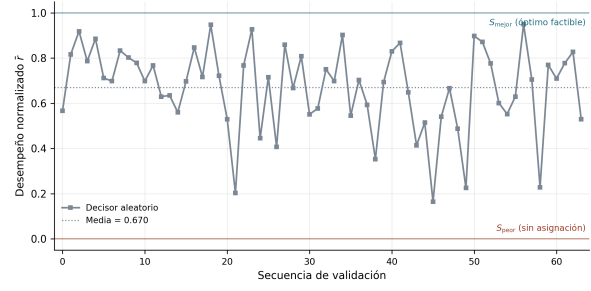
Donde S_i^{final} es la severidad de la ciudad i al terminar la secuencia; cuanto menor, mejor. Como esta cantidad depende de las severidades iniciales y de la longitud de cada secuencia, se normaliza con dos referencias: S_{peor} , la severidad acumulada si no se asigna ningún recurso, y S_{mejor} , la mínima severidad acumulada que se puede lograr con el presupuesto de la secuencia conociendo de antemano todas sus ciudades. Como la asignación de cada ciudad se fija al llegar y su evolución no depende de las demás, S_{mejor} se calcula de forma exacta por programación dinámica, como un problema de mochila acotada sobre asignaciones enteras de 0 a 10 por ciudad que suman a lo sumo 30. Ambas referencias se calculan al terminar la secuencia y no intervienen en las decisiones del agente. El *desempeño normalizado* de una secuencia es:

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}} \quad (4.5)$$

Donde S_{agente} es la severidad cruda que obtuvo el agente. Así, $\bar{r} = 0$ equivale a no asignar recursos y $\bar{r} = 1$ a la asignación óptima; como ningún agente puede gastar más que el presupuesto, $\bar{r} \leq 1$. Como referencia se evaluó un agente que elige acciones al azar sobre las mismas 64 secuencias (Figura 4.1): obtiene $\bar{r} = 0,670$ en promedio (desviación estándar 0,186, rango [0,164; 0,950]). Gastar el presupuesto sin criterio reduce parte de la severidad, pero queda lejos del óptimo en la mayoría de las secuencias.

4.4. Modelos individuales

La Tabla 4.1 lista los modelos individuales, el paquete que los implementa y el nombre con que se los menciona en el resto del texto; los ensambles se listan en la Tabla 4.5. Las

(a) Severidad final acumulada S_{cruda} .(b) Desempeño normalizado $\bar{r}$; las líneas horizontales marcan $\bar{r} = 0$ y $\bar{r} = 1$.**Figura 4.1:** Agente aleatorio (*decisor aleatorio* en la figura) en las 64 secuencias de evaluación. La línea punteada indica la media de cada serie.

figuras generadas por el código usan el nombre del paquete.

Tabla 4.1: Modelos individuales, paquete que los implementa y nombre usado en el texto. Q-Learning base se incluye sólo como referencia sin optimización.

Paquete	Nombre en el texto	Descripción
pes_base	Q-Learning base	Q-Learning tabular sin optimizar.
pes_q1	Q-Learning	Q-Learning tabular optimizado.
pes_dq1	Double Q-Learning	Double Q-Learning tabular con PBRS.
pes_dqn	DQN	Double DQN con red densa.
pes_rdn	DQN recurrente	Double DQN con codificador LSTM.
pes_a2c	A2C	Actor-crítico.
pes_trf	Transformer	Double DQN con codificador Transformer causal.

4.4.1. Modelos tabulares

Los tres modelos tabulares guardan $Q(s, a)$ en una tabla de $31 \times 11 \times 10 \times 11 = 37\,510$ valores, indexada por los enteros (R, t, S) y la acción. Las tablas se inicializan con valores de una distribución uniforme $\mathcal{U}(-1, 1)$, las acciones que exceden los recursos se enmascaran y, durante el entrenamiento, se elige la acción con una política ε -greedy. La actualización de Q-Learning es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.6)$$

Salvo en el estado terminal, donde $Q(s, a) \leftarrow r$. En la evaluación se elige la acción factible de mayor valor. La tabla sólo tiene filas para severidades de 0 a 9: si la severidad es mayor, el índice se recorta a 9. Como el entrenamiento sólo presenta severidades de 2 a 8, las filas de $S = 0, 1$ y 9 nunca se actualizan y conservan sus valores iniciales aleatorios en las tres tablas entrenadas.

4.4.1.1 Q-Learning base y Q-Learning

Ambos aplican la Ecuación (4.6) con una exploración ε que decrece linealmente desde ε_0 hasta $\varepsilon_{\min}$. Q-Learning base usa los valores originales del repositorio PES; Q-Learning usa los hiperparámetros elegidos por optimización bayesiana (Tabla 4.2).

4.4.1.2 Double Q-Learning

Mantiene dos tablas Q_A y Q_B (Ecuación (2.4)). En cada paso se actualiza una sola, elegida con probabilidad $1/2$, y la política final es $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$. La exploración se mantiene en ε_0 durante un calentamiento y luego decae exponencialmente:

$$\lambda = \left(\frac{\varepsilon_{\min}}{\varepsilon_0} \right)^{1/((q-w)N)} \quad \varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \lambda^{t-wN}) \quad \text{para } t \geq wN \quad (4.7)$$

Donde t es el número de episodio, N el total de episodios, w la fracción de episodios de calentamiento, q la fracción en la que se alcanza $\varepsilon_{\min}$ y λ el factor de decaimiento por episodio. Además usa PBRS con $\Phi(s) = -\sum_i S_i$, la suma de las severidades actuales de las ciudades con signo negativo, de modo que la recompensa con que se actualiza la tabla es:

$$\tilde{r} = r + \kappa [\gamma \Phi(s') - \Phi(s)] \quad (4.8)$$

Donde κ es el coeficiente de PBRS y s' el estado siguiente.

Tabla 4.2: Hiperparámetros de los modelos tabulares. Los de Q-Learning y Double Q-Learning son los del mejor ensayo de su optimización bayesiana.

	Q-Learning base	Q-Learning	Double Q-Learning
Tasa de aprendizaje α	0,2	0,2862	0,1132
Descuento γ	0,9	0,8587	0,9777
$\varepsilon_0 / \varepsilon_{\min}$	0,8 / 0	0,6813 / 0,0437	0,5998 / 0,0327
Decaimiento de ε	lineal	lineal	exponencial
Calentamiento w	—	—	0,0260
Fracción q	—	—	0,6430
PBRS κ	—	—	0,000392
Episodios	1 000 000	550 000	360 000
Ensayos de optimización	—	100	100

4.4.2. Modelos profundos

4.4.2.1 Entrada y configuración común

Los cuatro modelos profundos reciben el estado escalado por las cotas del entorno, $s = [R/30, t/10, S/9] \in [0, 1]^3$, para que las tres componentes tengan rangos comparables en la entrada de la red. Es una función fija de las cotas del MDP, que se aplica igual en entrenamiento, evaluación y ensambles. Con severidades mayores que 9, la entrada $S/9$ de DQN, DQN recurrente y Transformer simplemente supera 1; la evaluación de A2C, en cambio, recorta la severidad a 9 antes de escalarla. Todos usan el optimizador Adam [35], inicialización Glorot uniforme [36] y recorte de la norma del gradiente. La semilla de entrenamiento de cada modelo es $42 + i + 1$, con i el índice del mejor ensayo de su optimización.

DQN, DQN recurrente y Transformer comparten además la regla de aprendizaje. Usan el blanco Double DQN (Ecuación (2.6)) con enmascarado de acciones no factibles y pérdida de Huber, y exploran con una política ε -greedy según el esquema de la Ecuación (4.7). Empiezan a aprender cuando el *buffer* alcanza una fracción f de su capacidad y usan PBRS cuando $\kappa > 0$. Se diferencian en cómo procesan la entrada y en sus hiperparámetros (Tabla 4.3).

4.4.2.2 DQN

Red densa $\text{Input}(3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(11)$, que produce un valor Q por acción a partir del estado actual, con repetición de experiencia [9] y red objetivo [8].

4.4.2.3 DQN recurrente

Sigue el esquema DRQN [13]. La red recibe los últimos $W = 6$ estados escalados $(s_{t-5}, \dots, s_t)$, completados con ceros al principio de cada secuencia. Una LSTM de 64 unidades [12] recorre la ventana y su último estado oculto h_t (Ecuación (2.7)) pasa por dos capas densas de 96 unidades con ReLU y una salida de 11 valores Q . Cada ventana se procesa de forma independiente: el estado oculto no se conserva entre decisiones y el *buffer* guarda la ventana completa de cada transición. La búsqueda bayesiana incluyó W , el ancho de la LSTM y el de las capas densas, pero el entrenamiento final sólo toma del

mejor ensayo las capas densas y los hiperparámetros de entrenamiento: $W = 6$ y las 64 unidades de la LSTM son los valores fijos de la configuración del paquete (el mejor ensayo usaba 32 unidades).

4.4.2.4 Transformer

Recibe la misma ventana de $W = 6$ estados. Cada estado se proyecta a dimensión $d_{\text{model}} = 32$ y se le suma un vector de posición fijo, inicializado con Glorot uniforme a partir de la semilla y no entrenado. Cada uno de los $L = 2$ bloques tiene dos subcapas: atención causal (Ecuación (2.10)) con $h = 4$ cabezas de dimensión 16 y una capa *feed-forward* de 64 unidades. Cada subcapa tiene conexión residual y va precedida de normalización de capa (disposición *Pre-LN* [28]). No se usa *dropout*. La representación de la última posición pasa por una capa densa de 32 unidades con ReLU y una salida de 11 valores Q . La búsqueda bayesiana incluyó la ventana W y la arquitectura del codificador (d_{model} , número de cabezas y de bloques, dimensión de las claves, ancho de la capa *feed-forward* y *dropout*), pero el entrenamiento final sólo toma del mejor ensayo la capa densa de salida y los hiperparámetros de entrenamiento. La ventana y el codificador descritos son los valores fijos de la configuración del paquete; el mejor ensayo usaba $W = 3$, $d_{\text{model}} = 16$, 8 cabezas, 4 bloques, una capa *feed-forward* de 128 unidades y *dropout* de 0,15.

4.4.2.5 A2C

Actor y crítico son redes separadas: $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(11, \text{softmax})$ para el actor, que produce directamente una distribución sobre las 11 acciones, e $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(1)$ para el crítico. La ventaja se estima con GAE (Ecuación (2.9)) y se estandariza por lote; el gradiente de política (Ecuación (2.8)) y la entropía se calculan sobre la distribución restringida a las acciones factibles [15]. La Tabla 4.4 resume sus hiperparámetros.

4.5. Ensamblables

Los ensambles no se entrenan: cargan los modelos ya entrenados de DQN, DQN recurrente y Transformer y combinan sus salidas en cada decisión. Cinco de las seis variantes suman el actor de A2C como cuarto miembro. La Tabla 4.5 resume qué hace cada regla; el resto

Tabla 4.3: Arquitectura e hiperparámetros de DQN, DQN recurrente y Transformer, tal como se entrenaron los modelos desplegados. Entre paréntesis, el número de transiciones que corresponde a la fracción f . En DQN, todas las filas vienen del mejor ensayo de la optimización bayesiana. En DQN recurrente y Transformer, la entrada, los parámetros entrenables y los ensayos de optimización corresponden a la arquitectura fija de `config/CONFIG.py` del paquete; el resto de las filas (tasa de aprendizaje, γ , ε , fracciones w/q , lote, *buffer*, sincronización, recorte del gradiente, PBRS, inicio del aprendizaje y episodios) sí vienen del mejor ensayo de cada paquete (véase §4.4.2 para la discrepancia entre la arquitectura del mejor ensayo y la desplegada).

	DQN	DQN recurrente	Transformer
Entrada	estado actual	ventana de 6 estados	ventana de 6 estados
Parámetros entrenables	5 131	34 027	27 019
Tasa de aprendizaje	0,001508	0,002415	0,000215
Descuento γ	0,9634	0,9719	0,9234
ε_0 / $\varepsilon_{\min}$	0,9627 / 0,0691	0,8883 / 0,0887	0,8651 / 0,0838
Fracciones w / q	0,2779 / 0,6290	0,1237 / 0,5990	0,2428 / 0,5333
Tamaño de lote	128	64	128
Capacidad del <i>buffer</i>	20 000	60 000	30 000
Sincronización C (pasos)	1 000	2 000	500
Recorte del gradiente	3,953	3,475	4,170
PBRS κ	0,0226	0,00263	0
Inicio del aprendizaje f	0,1615 (3 230)	0,2021 (12 128)	0,1217 (3 650)
Episodios	40 000	30 000	30 000
Ensayos de optimización	47	20	10

Tabla 4.4: Hiperparámetros de A2C (mejor ensayo de 100 de la optimización bayesiana).

Tasa de aprendizaje inicial (actor / crítico)	0,000648 / 0,004299
Decaimiento de la tasa	coseno, hasta el 23,75 % del valor inicial
Descuento γ	0,8545
Coefficiente de entropía β_H	0,00528
λ de GAE	0,9135
Recorte del gradiente (ambas redes)	1,200
PBRS κ	0,1527
Penalización de gasto	$r \leftarrow r - 0,01039 a$
Sesgo inicial del logit de $a = 10$	-1,389
Episodios	125 000

de la sección las define con precisión, con una notación común.

4.5.1. Notación común

En el estado s , con R recursos disponibles, para cada miembro k se calculan:

- $p_k(a \mid s)$: su distribución sobre las acciones factibles $a \leq R$. Para DQN, DQN recurrente y Transformer se obtiene aplicando a sus valores $Q_k(s, a)$ la *softmax* de temperatura τ (Ecuación (2.13)) y renormalizando sobre las acciones factibles; para A2C es la salida del actor, renormalizada del mismo modo.

Tabla 4.5: Ensamblas evaluados, paquete que los implementa y qué hace cada regla de combinación.

Ensamble	Paquete	Cómo decide
Ensamble ponderado	<code>pes_ens</code>	Promedia las distribuciones de DQN, DQN recurrente y Transformer con pesos que crecen con la confianza; el Transformer tiene el mayor peso. Agrega un <i>prior</i> de severidad y una cota de seguridad.
Voto suave	<code>pes_ens_sprb</code>	Promedia las distribuciones de los cuatro miembros, ponderadas por peso y confianza.
Voto por acción	<code>pes_ens_accq</code>	Cada miembro vota por su acción preferida; los empates se resuelven con los valores Q .
Consenso	<code>pes_ens_consensus</code>	Voto ponderado más un premio por acuerdo y una penalización por desacuerdo entre miembros confiados.
Consenso con prior	<code>pes_ens_consensus_prior</code>	Como el consenso, pero sumando distribuciones y mezclando el resultado con un <i>prior</i> de severidad.
Compuerta del Transformer	<code>pes_ens_trf_guard</code>	Sigue al Transformer salvo cuando su confianza es baja; entonces usa un voto suave de respaldo.

- $a_k = \arg \max_a p_k(a \mid s)$: la acción que prefiere.
- $c_k = 1 - H_{\text{norm}}(p_k)$: su confianza en la decisión (Ecuación (2.12)), entre 0 y 1. En esta sección la entropía se normaliza por el logaritmo del número de acciones factibles y no por $\log |\mathcal{A}|$; la excepción es el ensamble ponderado, que normaliza por $\log 11$.
- w_k : su peso base, un parámetro fijo de cada ensamble.
- $\tilde{w}_k = w_k c_k^\rho$: su peso efectivo, que combina el peso base con la confianza; el exponente $\rho \geq 0$ regula cuánto influye la confianza ($\rho = 0$ la ignora).
- $\hat{Q}_k(a) = (Q_k(a) - \bar{Q}_k) / \text{sd}(Q_k)$: sus valores Q estandarizados, con $\bar{Q}_k$ y $\text{sd}(Q_k)$ la media y la desviación estándar de sus valores sobre las acciones factibles en el estado s (para A2C se estandarizan sus probabilidades). Permiten comparar modelos cuyos valores tienen escalas distintas.

Los parámetros de cada ensamble se listan en la Tabla 4.6. Los del ensamble ponderado son valores fijos de su configuración. Los de los otros cinco provienen de estudios de Optuna (TPE) sobre las 64 secuencias de evaluación. Sólo en el consenso con *prior* el estudio guardado corresponde a la configuración evaluada: su puntaje registrado (0,918)

coincide con la media que obtiene el ensamble en la referencia. En los otros cuatro, el archivo de parámetros no incluye el peso de A2C, que toma el valor por defecto 0,10, y el puntaje registrado por el estudio (entre 0,888 y 0,911) no coincide con la media evaluada (Tabla 5.3). Esos estudios se hicieron, por lo tanto, con una configuración de miembros distinta de la evaluada.

4.5.2. Ensamble ponderado

Combina DQN, DQN recurrente y Transformer ($K = 3$) con un voto suave cuyos pesos crecen con la confianza de cada miembro:

$$p_{\text{ens}}(a \mid s) = \frac{\eta(a) \sum_{k=1}^K w_k (0,1 + c_k) p_k(a \mid s)}{\sum_{a' \leq R} \eta(a') \sum_{k=1}^K w_k (0,1 + c_k) p_k(a' \mid s)} \quad \eta(a) = \begin{cases} 0,3 & a = 0, R > 0 \\ 1 & \text{en otro caso} \end{cases} \quad (4.9)$$

Donde w_k son los pesos base (0,18, 0,90 y 5,0 para DQN, DQN recurrente y Transformer, normalizados para que sumen 1). El término $0,1+c_k$ hace que el peso crezca con la confianza sin anularse cuando $c_k = 0$, y $\eta(a)$ reduce la probabilidad de no asignar nada mientras queden recursos. El denominador normaliza la distribución sobre las acciones factibles. Aquí la *softmax* usa $\tau = 15$. Luego esa distribución se mezcla con una distribución previa (*prior*) centrada en la severidad actual:

$$\pi_{\text{prior}}(a \mid s) \propto \exp\left[-\frac{(a - S)^2}{2\sigma_S^2}\right] \quad p'(a \mid s) \propto (1 - w_\pi) p_{\text{ens}}(a \mid s) + w_\pi \pi_{\text{prior}}(a \mid s) \quad (4.10)$$

Donde S es la severidad de la ciudad actual, $\sigma_S = 3$ el ancho del *prior* y $w_\pi = 0,17$ su peso en la mezcla; p' se renormaliza sobre las acciones factibles. La acción ejecutada es la de mayor probabilidad:

$$a_{\text{ens}} = \arg \max_{a \leq R} p'(a \mid s) \quad (4.11)$$

Por último, una cota de seguridad reemplaza esa acción por $\lfloor S/2 \rfloor$ cuando $S \geq 6$, la acción

elegida es menor que esa cota y la cota es factible. Como en A2C, la severidad de entrada se recorta a 9 antes de escalarla, tanto para los miembros como para el *prior*.

4.5.3. Voto suave

Promedia las distribuciones de los miembros con sus pesos efectivos y elige la acción más probable:

$$p_{\text{vs}}(a \mid s) = \frac{\sum_k \tilde{w}_k p_k(a \mid s)}{\sum_k \tilde{w}_k} \quad a_{\text{vs}} = \arg \max_a p_{\text{vs}}(a \mid s) \quad (4.12)$$

Donde la suma recorre los cuatro miembros. Es la única variante que ajusta la temperatura de la *softmax* ($\tau = 0,1044$); en las demás $\tau = 1$.

4.5.4. Voto por acción

Cada miembro vota por su acción preferida con su peso efectivo; si hay empate, gana la acción con mayor suma de valores estandarizados:

$$V(a) = \sum_k \tilde{w}_k \mathbf{1}[a_k = a] \quad T(a) = \sum_k \hat{Q}_k(a) \quad (4.13)$$

$$a_{\text{va}} = \arg \max_{a \in \mathcal{V}} T(a) \quad \mathcal{V} = \{a : V(a) = \max_{a'} V(a')\} \quad (4.14)$$

Donde $V(a)$ es el total de votos ponderados que recibe la acción a , $T(a)$ el criterio de desempate y $\mathcal{V}$ el conjunto de acciones más votadas.

4.5.5. Consenso

Parte del mismo voto ponderado y le suma un premio por acuerdo, una penalización por desacuerdo y los valores estandarizados de los miembros ponderados por su confianza:

$$G(a) = \sum_k \tilde{w}_k \mathbf{1}[a_k = a] + \beta_a \sum_{k: a_k = a} c_k - \beta_d \sum_{k: a_k \neq a} c_k + \sum_k \hat{Q}_k(a) c_k \quad (4.15)$$

$$a_{\text{cons}} = \arg \max_a G(a)$$

Donde $G(a)$ es el puntaje de la acción a . El segundo término suma la confianza de los miembros que prefieren a , multiplicada por el premio β_a . El tercero resta la confianza de los que prefieren otra acción, multiplicada por la penalización β_d . El cuarto agrega lo que cada miembro opina de a aunque no sea su preferida.

4.5.6. Consenso con *prior*

Reemplaza el voto duro del consenso por una suma de distribuciones, $\tilde{G}(a) = \sum_k \tilde{w}_k p_k(a | s)$, a la que suma los mismos términos de acuerdo y desacuerdo de la Ecuación (4.15). Luego multiplica $\tilde{G}(0)$ por 0,3 cuando quedan recursos, normaliza $\tilde{G}$ como una distribución y la mezcla con el *prior* de severidad de la Ecuación (4.10), con sus propios w_π y σ_S . La acción es la de mayor valor en esa mezcla, con la misma cota de seguridad que el ensamble ponderado. A diferencia de los demás, calcula la confianza aplicando la *softmax* a p_k en lugar de a los valores Q , lo que acerca la distribución a la uniforme y reduce c_k . Es la única variante en la que se optimizó el peso del actor de A2C; en las demás vale 0,10.

4.5.7. Compuerta del Transformer

Sigue al Transformer salvo cuando su confianza es baja. Para decidirlo calcula una compuerta logística:

$$g = \frac{1}{1 + e^{-\kappa_g (c_{\text{trf}} - \tau_g)}} \quad f(a) = \sum_k \tilde{w}_k p_k(a | s) \quad (4.16)$$

Donde c_{trf} es la confianza del Transformer, τ_g el umbral de confianza, κ_g la pendiente de la compuerta y $f(a)$ un voto suave de respaldo con todos los miembros. Si $g \geq 0,5$ se ejecuta la acción del Transformer, a_{trf} ; si no, la de mayor $f(a)$. Como $g \geq 0,5$ equivale a $c_{\text{trf}} \geq \tau_g$, el ensamble sigue al Transformer siempre que su confianza supere $\tau_g = 0,2248$.

Como las reglas no se entrenan junto con los miembros, las diferencias entre ensambles miden el efecto de la regla de combinación y de sus parámetros sobre los mismos modelos ya entrenados.

Tabla 4.6: Parámetros de los seis ensambles: temperatura τ , exponente de confianza ρ y pesos base de DQN (w_{DQN}), DQN recurrente (w_{rec}), Transformer (w_{trf}) y A2C (w_{A2C}). En el ensamble ponderado los pesos se multiplican por $0,1 + c_k$ en lugar de c_k^ρ y el actor de A2C no participa.

Ensamble	τ	ρ	w_{DQN}	w_{rec}	w_{trf}	w_{A2C}	Otros
Ponderado	15	—	0,18	0,90	5,0	—	$w_\pi = 0,17, \sigma_S = 3$
Voto suave	0,1044	0,4677	2,852	1,628	1,134	0,10	—
Voto por acción	1	1,005	2,374	0,722	1,008	0,10	—
Consenso	1	0,4026	2,995	0,320	1,695	0,10	$\beta_a = 0,1649,$ $\beta_d = 0,3647$
Consenso con prior	1	0,4180	2,778	2,465	2,840	0,1044	$\beta_a = 1,810,$ $\beta_d = 0,3462,$ $w_\pi = 0,2051,$ $\sigma_S = 0,9177$
Compuerta Transformer	1	1,896	1,124	2,852	2,196	0,10	$\tau_g = 0,2248,$ $\kappa_g = 3,964$

4.6. Optimización de hiperparámetros

Cada paquete optimizable tiene un módulo `optimize_*.py` que ejecuta un estudio de Optuna con TPE [20, 21]. En cada ensayo se entrena el agente desde cero con una configuración θ y se calcula el desempeño medio $\bar{r}$ (Ecuación (4.5)) sobre las 64 secuencias; el estudio busca $\theta^* = \arg \max_{\theta} \bar{r}(\theta)$. Los ensayos poco prometedores se interrumpen con `MedianPruner` y el estudio se guarda en una base SQLite, lo que permite reanudarlo. El modelo final se reentrena desde cero con θ^* y la semilla de su ensayo. El número de episodios forma parte del espacio de búsqueda, y el entrenamiento final usa el del mejor ensayo. Los números de ensayos y de episodios de cada modelo están en las Tablas 4.2, 4.3 y 4.4; A2C usó 100 ensayos. En Double Q-Learning, 86 de los 100 ensayos se interrumpieron por el `MedianPruner`.

En Q-Learning, Double Q-Learning, DQN y A2C, el modelo final reproduce el puntaje de su mejor ensayo. En el DQN recurrente y el Transformer, en cambio, el reentrenamiento conserva la arquitectura fija del paquete y no la del mejor ensayo. El DQN recurrente final obtiene 0,899, por debajo de los 0,926 de su mejor ensayo; el Transformer final obtiene 0,927, frente a 0,925.

4.7. Escenarios de generalización

Además de la condición de referencia (`sev_base`), que reproduce la distribución de entrenamiento, cada modelo se evalúa en 21 escenarios de generalización (Tabla 4.7), agrupados en cuatro familias:

- **Severidad (9).** Cambian la distribución de las severidades iniciales (uniforme, gaussianas de media baja, media y alta, Weibull, Beta sesgadas y bimodal). Uno de ellos, la *extrapolación de severidad* (`sev_extrapolate_high`), usa severidades de 10 a 12, por encima de la cota del entorno ($S_{\text{máx}} = 9$). Los otros ocho incluyen severidades de 0, 1 o 9, que están dentro del espacio de estados pero no aparecen en el entrenamiento (2 a 8).
- **Longitud (5).** Cambian la longitud de las secuencias (todas de 3, todas de 10, geométrica, Poisson). Uno de ellos, el de *secuencias largas* (`len_extrapolate_long`), usa secuencias de 11 a 20 pasos, por encima de la cota del entorno ($T_{\text{máx}} = 10$).
- **Conjunta (4).** Cambian severidad y longitud a la vez; la *extrapolación conjunta* (`joint_extrap_both`) lleva ambas por encima de sus cotas.
- **Estructural (3).** Cambian sólo la partición en bloques o el número de secuencias ($n = 128$ en uno de ellos), sin cambiar la distribución. Sirven de control: deben reproducir el resultado de la referencia.

En el resto del texto se usan los nombres en cursiva; los identificadores aparecen en la Tabla 4.7 y en las figuras. Se llama *fuera de rango* a los tres escenarios que superan las cotas del entorno.

Los escenarios de generalización se generan con la semilla 42, de modo que, dentro de cada escenario, todos los modelos enfrentan las mismas secuencias. La evaluación es sólo de inferencia: ningún modelo se reentrena. Para cada modelo y escenario se calcula la *degradación* $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{esc}}$, es decir, cuánto desempeño pierde respecto de la referencia (un valor negativo indica que mejora).

4.8. Protocolo de evaluación

1. **Optimización y reentrenamiento.** Cada modelo optimizable se ajusta con Optuna y se reentrena desde cero con los mejores hiperparámetros. Q-Learning base se entrena con sus valores fijos.
2. **Evaluación determinista.** Cada modelo se evalúa en los 22 escenarios (Apéndice B) eligiendo siempre la acción factible de mayor valor ($\varepsilon = 0$). Los modelos individuales y los ensambles se evalúan como dos grupos de seis modelos cada uno;

Tabla 4.7: Condición de referencia y 21 escenarios de generalización. Los escenarios con **extrap** en el nombre superan las cotas del entorno ($S \leq 9$, $T \leq 10$).

Familia	Identificador	Descripción
referencia	sev_base	Distribución de entrenamiento.
severidad	sev_uniform	$U\{0, \dots, 9\}$, sin estructura modal.
severidad	sev_gauss_low	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	sev_gauss_mid	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	sev_gauss_high	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	sev_weibull	Weibull($k = 1, 5$), cola superior pesada.
severidad	sev_beta_lowskew	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	sev_beta_highskew	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	sev_bimodal	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	sev_extrapolate_high	$U\{10, 11, 12\}$, fuera de rango.
longitud	len_all_short	Todas las secuencias de longitud 3.
longitud	len_all_long	Todas las secuencias de longitud 10.
longitud	len_geometric	$\text{Geom}(p = 0, 2)$, recortada a $[3, 10]$.
longitud	len_poisson	$\text{Poisson}(\lambda = 5)$, recortada a $[3, 10]$.
longitud	len_extrapolate_long	$U\{11, \dots, 20\}$, fuera de rango.
conjunta	joint_high_long	Gaussiana alta $\times$ todas largas.
conjunta	joint_low_short	Gaussiana baja $\times$ todas cortas.
conjunta	joint_uniform_geom	Uniforme $\times$ geométrica.
conjunta	joint_extrap_both	Severidad y longitud fuera de rango.
estructural	struct_few_long_blocks	4 bloques $\times$ 16 secuencias.
estructural	struct_many_short_blocks	16 bloques $\times$ 4 secuencias.
estructural	struct_more_total	8 bloques $\times$ 16 secuencias ($n = 128$).

Q-Learning base se agrega como referencia.

3. **Comparación estadística** (Sección 2.11). Para cada modelo y escenario se calculan la media y la desviación estándar de $\bar{r}$, la degradación, y el p de Welch y la d de Cohen frente a su propia referencia. También se calcula la divergencia KL entre su distribución de acciones y la de la referencia. Para cada par de modelos de un mismo grupo se calculan la d de Cohen, el p de Welch y la KL simetrizada entre sus histogramas de desempeño, juntando las secuencias de los 21 escenarios de generalización. Como todos los modelos enfrentan las mismas secuencias, la prueba de Welch se usa como contraste de referencia; un contraste estrictamente pareado requeriría analizar las diferencias secuencia por secuencia.

5. Resultados

Este capítulo responde las dos preguntas de la Sección 1.3: cuál es el mejor modelo individual y si algún ensamble lo supera. Todos los valores son el desempeño normalizado $\bar{r}$ (Ecuación (4.5)) promediado sobre las secuencias de cada escenario. *Referencia* designa la condición de entrenamiento y *generalización*, la media en los 21 escenarios de generalización (Sección 4.7). Los mapas estadísticos de apoyo están en el Apéndice C.

5.1. Mejor modelo individual

El Transformer es el mejor modelo individual, tanto en la referencia como en generalización (Tabla 5.1). En generalización obtiene 0,930; el resto de los modelos optimizados queda entre 0,871 y 0,899. También tiene la menor dispersión entre secuencias y la mayor media en 16 de los 22 escenarios (Figura 5.1; con dos decimales, dos celdas aparecen empatadas). La media de generalización incluye los tres escenarios estructurales, que reproducen la referencia, y escenarios más fáciles que ella, como las secuencias cortas; por eso conviene leerla junto con el peor escenario. En ese criterio el Transformer no es el que menos pierde: cae 0,067 respecto de su referencia, frente a 0,053 de DQN y 0,061 de A2C.

Tabla 5.1: Desempeño normalizado medio $\bar{r}$ de cada modelo individual en la referencia (con su desviación estándar entre secuencias, σ), media en los 21 escenarios de generalización y peor escenario.

Modelo	Referencia	σ	Generalización	Peor escenario
Transformer	0,927	0,045	0,930	Secuencias largas (0,860)
DQN	0,894	0,055	0,899	Secuencias largas (0,841)
A2C	0,887	0,063	0,896	Secuencias largas (0,826)
DQN recurrente	0,899	0,049	0,889	Severidad fuera de rango (0,831)
Double Q-Learning	0,896	0,048	0,877	Severidad fuera de rango (0,785)
Q-Learning	0,887	0,061	0,871	Severidad fuera de rango (0,762)
Q-Learning base	0,871	0,074	0,851	Severidad fuera de rango (0,627)

La ventaja también puede expresarse como distancia al óptimo. Si $1 - \bar{r}$ mide cuánto le falta al agente para igualar la asignación óptima, la reducción relativa de esa distancia para un modelo m respecto de Q-Learning es:

$$\Delta_{\text{rel}}(m) = \frac{(1 - \bar{r}_{\text{ql}}) - (1 - \bar{r}_m)}{1 - \bar{r}_{\text{ql}}} = \frac{\bar{r}_m - \bar{r}_{\text{ql}}}{1 - \bar{r}_{\text{ql}}} \quad (5.1)$$

En la referencia, el Transformer reduce en alrededor de un 36 % la distancia que deja Q-Learning.

5.1.1. Desempeño por familia de escenarios

Cada familia falla en un tipo distinto de escenario (Figura 5.1; la diferencia de cada celda con la primera columna es la degradación). Los modelos tabulares caen por debajo de 0,8 en la extrapolación de severidad. Su tabla recorta esas severidades a $S = 9$, una fila que el entrenamiento nunca visitó, de modo que deciden con los valores iniciales aleatorios de la tabla (Sección 4.4.1). A2C también recibe la severidad recortada a 9, pero, como red, extiende a esa entrada lo aprendido para severidades cercanas y obtiene 0,929.

Las redes mantienen su desempeño en ese escenario: el Transformer queda a menos de 0,005 del óptimo ($\bar{r} = 0,996$ en la extrapolación de severidad y 0,997 en la conjunta). Eso no basta para concluir que generaliza mejor allí que en la referencia: con severidades de 10 a 12, una política que asigna mucho a cada ciudad puede acercarse al óptimo, y A2C también supera en ese escenario su propia referencia. Su punto débil son las secuencias más largas que las de entrenamiento. El DQN recurrente es la excepción: falla más con severidades fuera de rango que con secuencias largas.

Los tres escenarios estructurales reproducen el valor de la referencia, como se espera de un control.

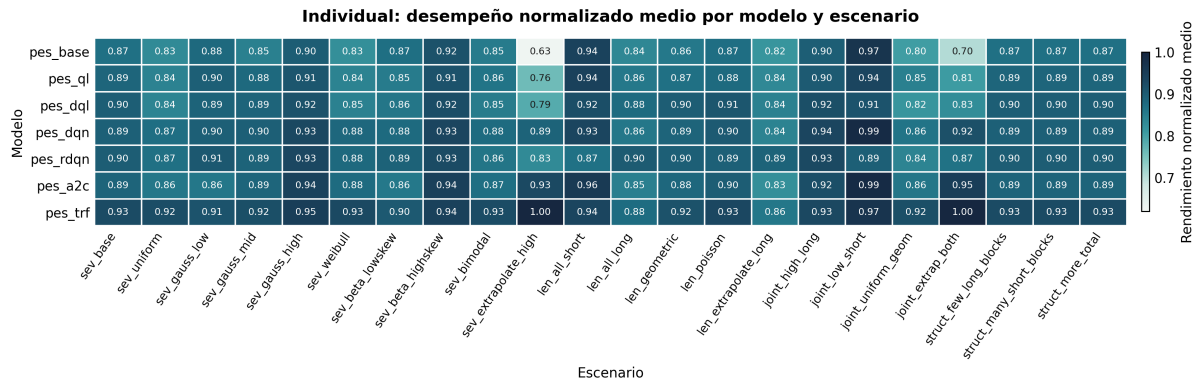


Figura 5.1: Modelos individuales: desempeño medio $\bar{r}$ por modelo y escenario. La primera columna es la referencia.

5.1.2. Tamaño de la ventaja del Transformer

Para saber si la diferencia de medias es grande en relación con la variabilidad entre secuencias, la Tabla 5.2 compara al Transformer con las dos redes que usan su misma regla de aprendizaje, DQN y DQN recurrente, y con Q-Learning. Cada comparación junta las secuencias de los 21 escenarios de generalización.

Tabla 5.2: Comparación del Transformer con otros modelos sobre los 21 escenarios de generalización. d : diferencia de medias en desviaciones estándar (positiva si gana el Transformer); $\log_{10} p$: significancia de la prueba de Welch; $D_{\text{KL}}^{\text{sym}}$: diferencia entre los histogramas de desempeño de ambos modelos (Sección 2.11).

Comparación	d de Cohen	$\log_{10} p$	$D_{\text{KL}}^{\text{sym}}$
Transformer vs Q-Learning	0,78	−87,8	0,49
Transformer vs DQN recurrente	0,57	−48,9	0,34
Transformer vs DQN	0,51	−39,2	0,17

La medida principal es la d de Cohen, que mide el tamaño de la ventaja. Es de tamaño medio frente a las otras dos redes y cercana a grande frente a Q-Learning. El $\log_{10} p$ es muy negativo en las tres comparaciones, pero debe leerse con cautela. Las secuencias no son independientes entre sí: se generan con la misma semilla y son las mismas para ambos modelos. Además, con más de mil secuencias por modelo incluso diferencias moderadas dan valores p muy pequeños. La divergencia KL indica que la distribución de desempeños del Transformer se parece a la de DQN y se aleja más de la de Q-Learning. La comparación entre todos los pares de modelos está en el Apéndice (Figura C.3).

5.1.3. Resultados por modelo en la referencia

Las Figuras 5.2–5.8 muestran, para cada modelo individual, el desempeño de cada una de las 64 secuencias de la referencia, su distribución, los estadísticos por bloque y un resumen descriptivo. En ellas cada modelo aparece con el nombre de su paquete (Tabla 4.1).

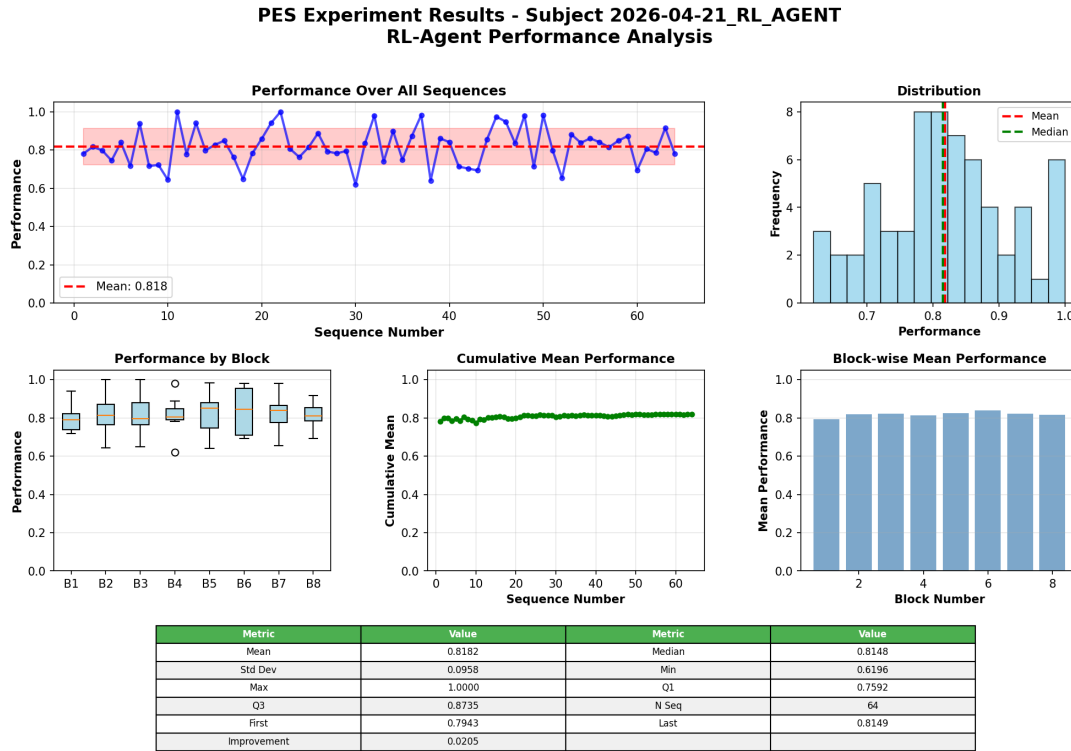


Figura 5.2: Resultados por secuencia de Q-Learning base (pes_base) en la referencia.

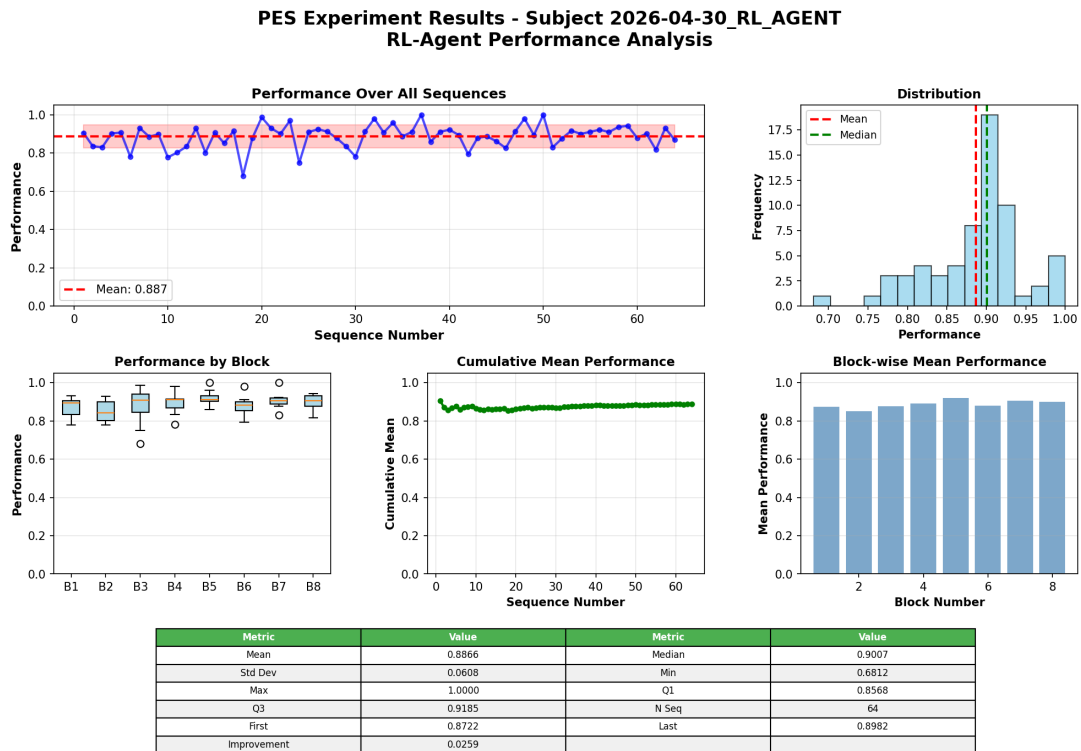


Figura 5.3: Resultados por secuencia de Q-Learning (pes_q1) en la referencia.

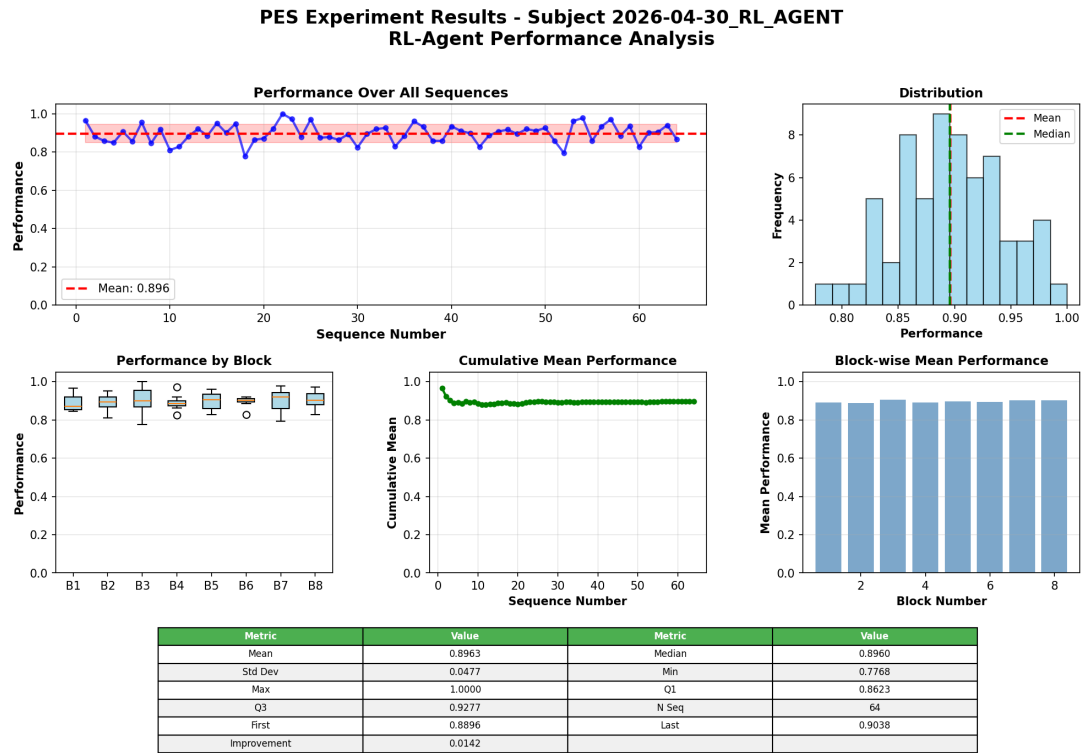


Figura 5.4: Resultados por secuencia de Double Q-Learning (pes_dql) en la referencia.

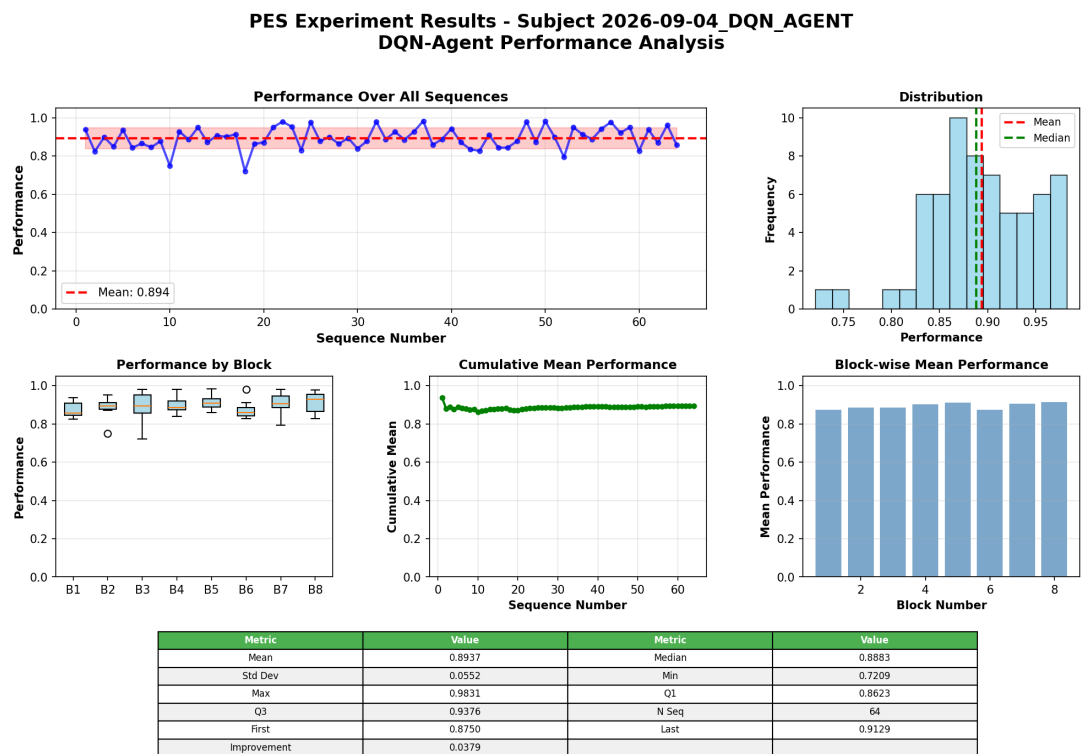


Figura 5.5: Resultados por secuencia de DQN (pes_dqn) en la referencia.

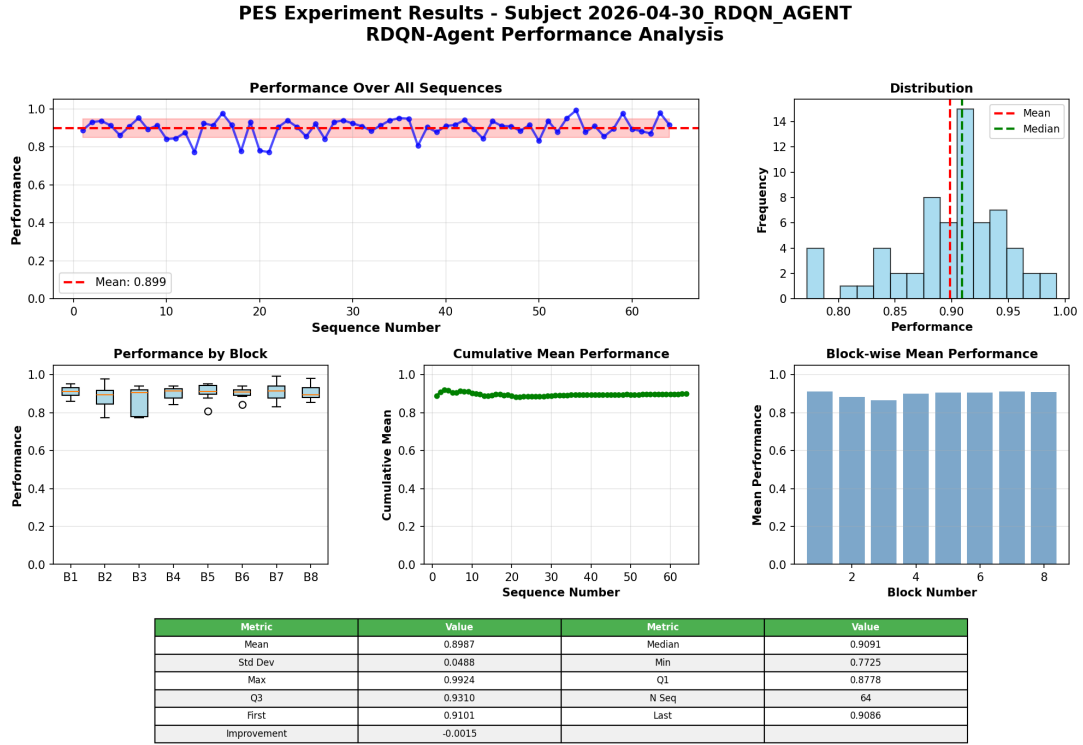


Figura 5.6: Resultados por secuencia del DQN recurrente (pes_rdqn) en la referencia.

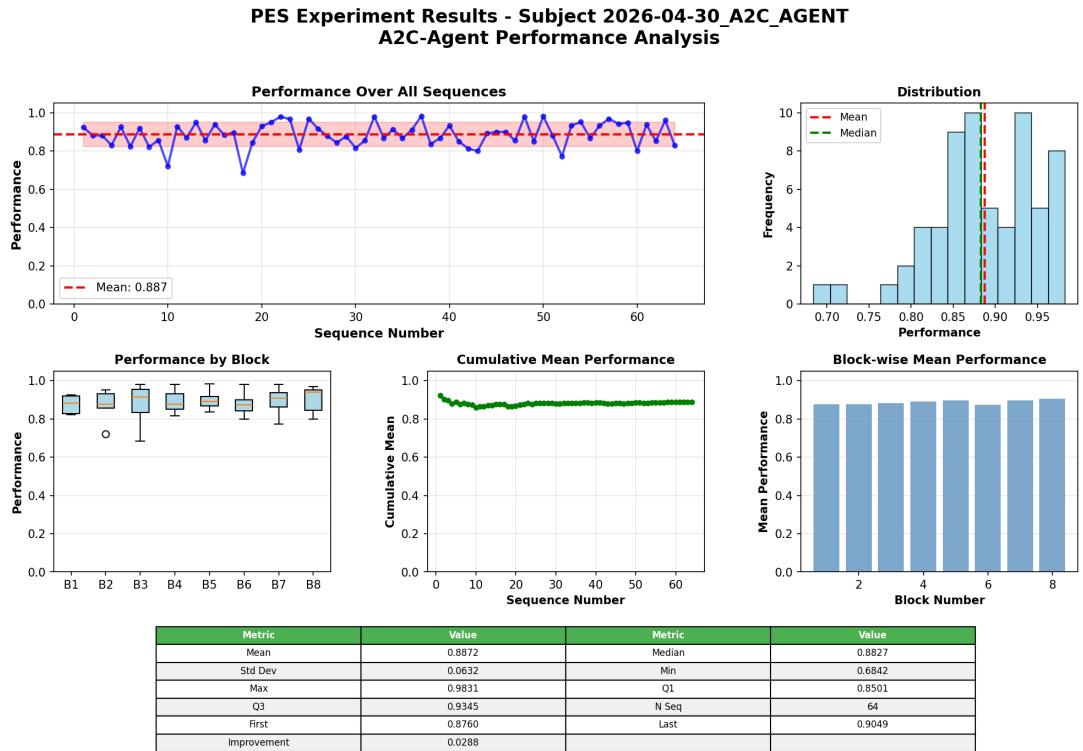


Figura 5.7: Resultados por secuencia de A2C (pes_a2c) en la referencia.

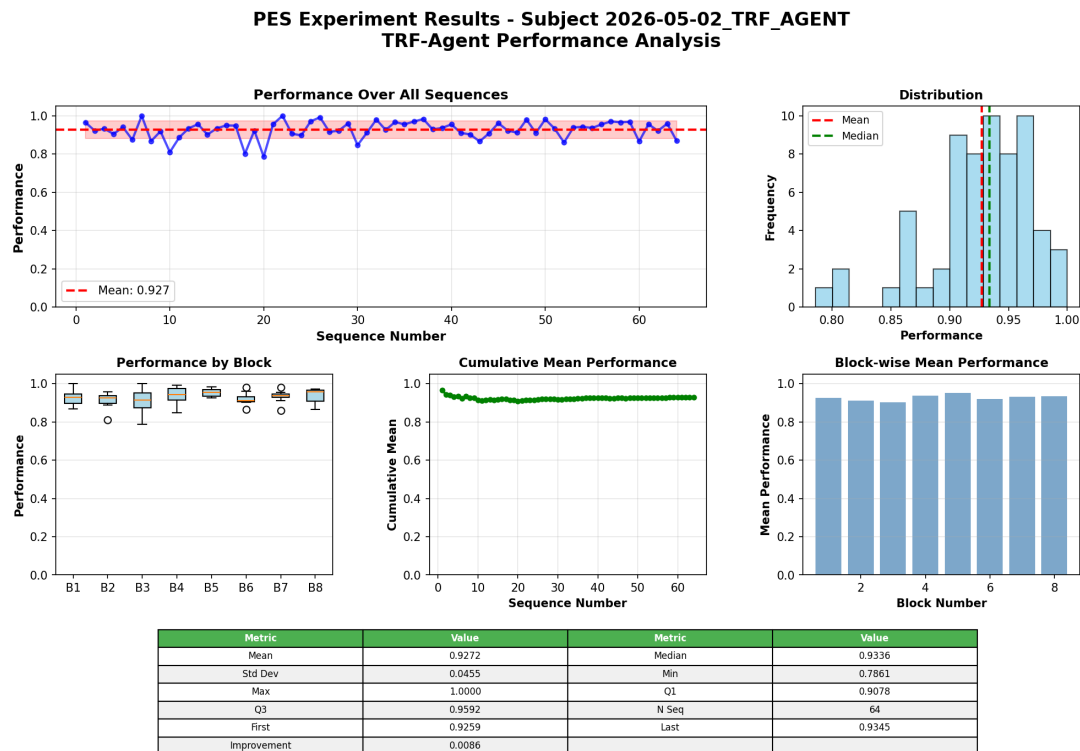


Figura 5.8: Resultados por secuencia del Transformer (`pes_trf`) en la referencia.

5.1.4. Curvas por escenario

La Figura 5.9 muestra el desempeño de cada secuencia en seis escenarios de generalización: tres de severidad, dos de longitud y uno conjunto. En cada panel las secuencias se ordenan de menor a mayor $\bar{r}$: una curva por encima de otra indica mejor desempeño en toda la distribución, y el tramo inicial muestra los peores casos. En los escenarios de severidad y en el de longitudes Poisson, el Transformer queda por encima o al nivel del resto, con la mayor ventaja en los peores casos de la distribución bimodal. Con secuencias largas lo supera el DQN recurrente, y con alta severidad y secuencias largas las curvas casi se superponen.

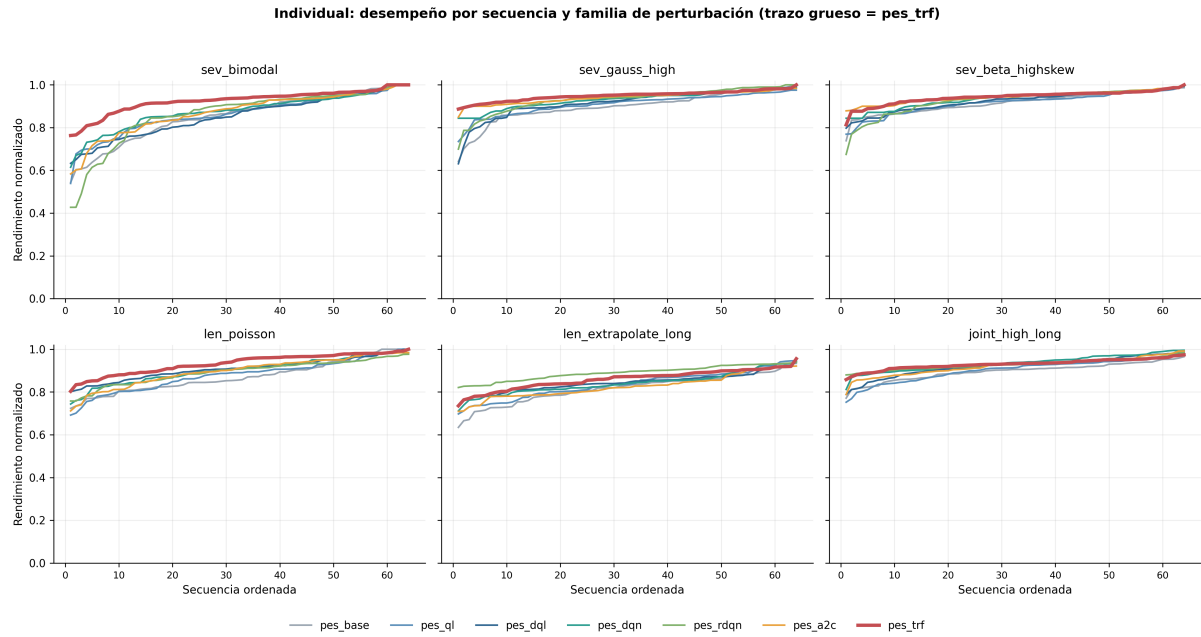


Figura 5.9: Desempeño por secuencia, ordenado de menor a mayor, en seis escenarios de generalización. Fila superior: severidad; fila inferior: longitud y conjunta. El trazo grueso corresponde al Transformer.

5.2. Ensamblas

El ensemble ponderado es el único que supera al Transformer (Tabla 5.3). En generalización obtiene 0,939, frente a 0,930 del Transformer solo, y su peor escenario queda 0,040 por encima del peor del Transformer. En la referencia reduce en alrededor de un 45 % la distancia que deja Q-Learning (Ecuación (5.1)), frente al 36 % del Transformer.

Tabla 5.3: Desempeño de los ensambles en la referencia, media en los 21 escenarios de generalización y mayor degradación respecto de la referencia.

Ensamble	Referencia	Generalización	Mayor degradación
Ensamble ponderado	0,937	0,939	0,037
Compuerta del Transformer	0,928	0,931	0,067
Consenso con prior	0,918	0,923	0,042
Consenso	0,893	0,904	0,063
Voto suave	0,914	0,902	0,054
Voto por acción	0,914	0,901	0,055

Los otros cinco ensambles no superan al Transformer. La compuerta del Transformer lo iguala, con resultados casi idénticos a los del Transformer solo. Los dos consensos quedan por debajo. El voto suave y el voto por acción, que sólo promedian o votan, obtienen las medias más bajas y tienen su peor escenario en la extrapolación de severidad, el escenario

en que el Transformer casi alcanza el óptimo.

Como en los modelos individuales, cada ensamble falla en un tipo distinto de escenario (Figura 5.10). La longitud es el cambio más desfavorable para la compuerta del Transformer y los dos consensos; el voto suave y el voto por acción pierden más con la severidad.

5.2.1. Cuánto cambian las reglas fijas la decisión

Para saber cuánto pesan en la decisión el Transformer, el *prior* de severidad y la cota de seguridad, se repitió la evaluación de los tres ensambles que usan esas reglas (Apéndice B). En cada decisión con recursos disponibles se registró la acción antes y después de cada regla y la acción que prefiere el Transformer. La repetición reproduce las medias del benchmark con una diferencia máxima de 0,002. La Tabla 5.4 resume los resultados.

Tabla 5.4: Porcentaje de las decisiones con recursos disponibles en que ocurre cada evento, en la referencia y en el conjunto de los 21 escenarios de generalización.

Ensamble	Evento	Referencia	Generalización
Compuerta del Transformer	Sigue al Transformer ($g \geq 0,5$)	96,9	95,0
	Acción final distinta de la del Transformer	0,8	2,6
Consenso con prior	El <i>prior</i> cambia la acción votada	7,7	9,0
	La cota cambia la acción	0,0	0,6
	Acción final distinta de la del Transformer	61,3	56,3
Ensamble ponderado	El <i>prior</i> cambia la acción votada	40,9	32,5
	La cota cambia la acción	0,0	0,3
	Acción final distinta de la del Transformer	61,2	48,1

La compuerta sigue al Transformer en casi todas las decisiones, lo que explica que sus resultados sean casi idénticos a los del Transformer solo. En el ensamble ponderado ocurre lo contrario: aunque el Transformer tiene el 82 % del peso base, la acción final difiere de la suya en el 61 % de las decisiones de la referencia y en el 48 % de las de generalización. Con $\tau = 15$ las distribuciones de los tres miembros quedan casi planas. En la referencia, la acción más probable de cada uno tiene en promedio probabilidad 0,19 y supera a la segunda por sólo 0,02–0,03. Por eso el voto ya difiere de la acción del Transformer en el 31 % de las decisiones, y el *prior* cambia la acción votada en el 41 %. La cota de seguridad casi nunca actúa. En la extrapolación de severidad y en la conjunta, en cambio, todas las

decisiones del ensamble ponderado coinciden con las de su miembro Transformer, que allí recibe la severidad recortada a 9.

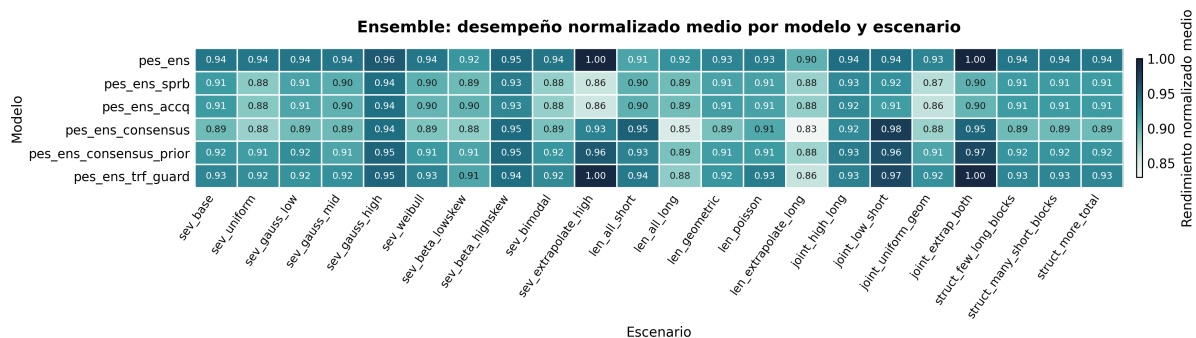


Figura 5.10: Ensamblas: desempeño medio $\bar{r}$ por ensamble y escenario. La primera columna es la referencia.

La Figura 5.11 muestra el detalle por secuencia en los tres escenarios fuera de rango. El ensamble ponderado alcanza el óptimo en todas las secuencias de la extrapolación de severidad y de la conjunta ($\bar{r} = 1, \sigma = 0$), y tiene la mayor media con secuencias largas. El voto suave y el voto por acción, en cambio, se separan en dos grupos en la extrapolación de severidad: en cerca del 40 % de las secuencias quedan por debajo de 0,8, mientras que en el resto se acercan a los demás.

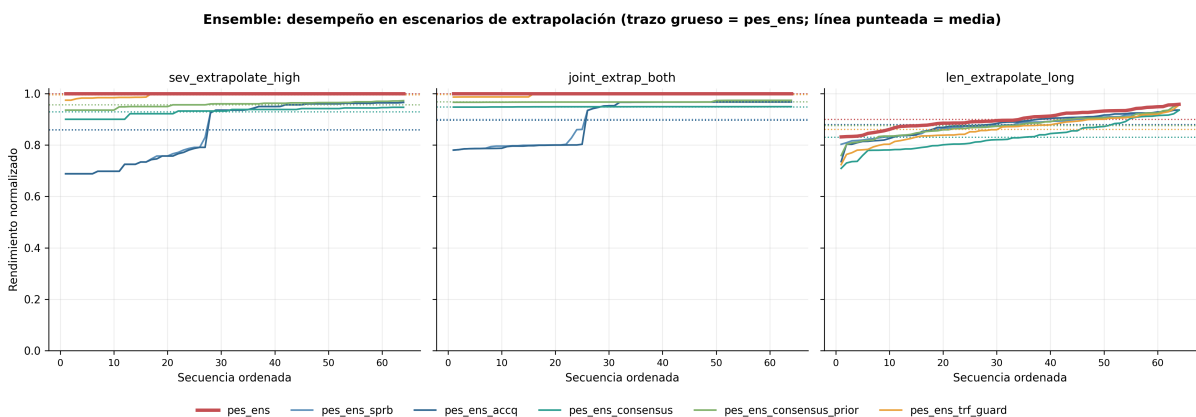


Figura 5.11: Desempeño por secuencia, ordenado de menor a mayor, de los seis ensambles en los tres escenarios fuera de rango; el trazo grueso corresponde al ensamble ponderado y las líneas punteadas a la media de cada ensamble.

Para saber si la ventaja del ensamble ponderado sobre los demás es real, la Tabla 5.5 lo compara con el consenso y con la compuerta del Transformer. Frente al consenso la ventaja es de tamaño medio ($d = 0,62$). Frente a la compuerta, que se comporta casi como el Transformer solo, es pequeña ($d = 0,17$). Las comparaciones entre todos los pares están en el Apéndice (Figura C.5).

La Figura 5.12 muestra el detalle por secuencia del ensamble ponderado, que combina la

Tabla 5.5: Comparación del ensemble ponderado con otros ensambles sobre los 21 escenarios de generalización; columnas como en la Tabla 5.2.

Comparación	d de Cohen	$\log_{10} p$	D_{KL}^{sym}
Ponderado vs consenso	0,62	−55,6	0,25
Ponderado vs compuerta del Transformer	0,17	−5,1	0,03

mayor media de todos los sistemas evaluados con la menor dispersión entre secuencias ($\sigma = 0,035$).

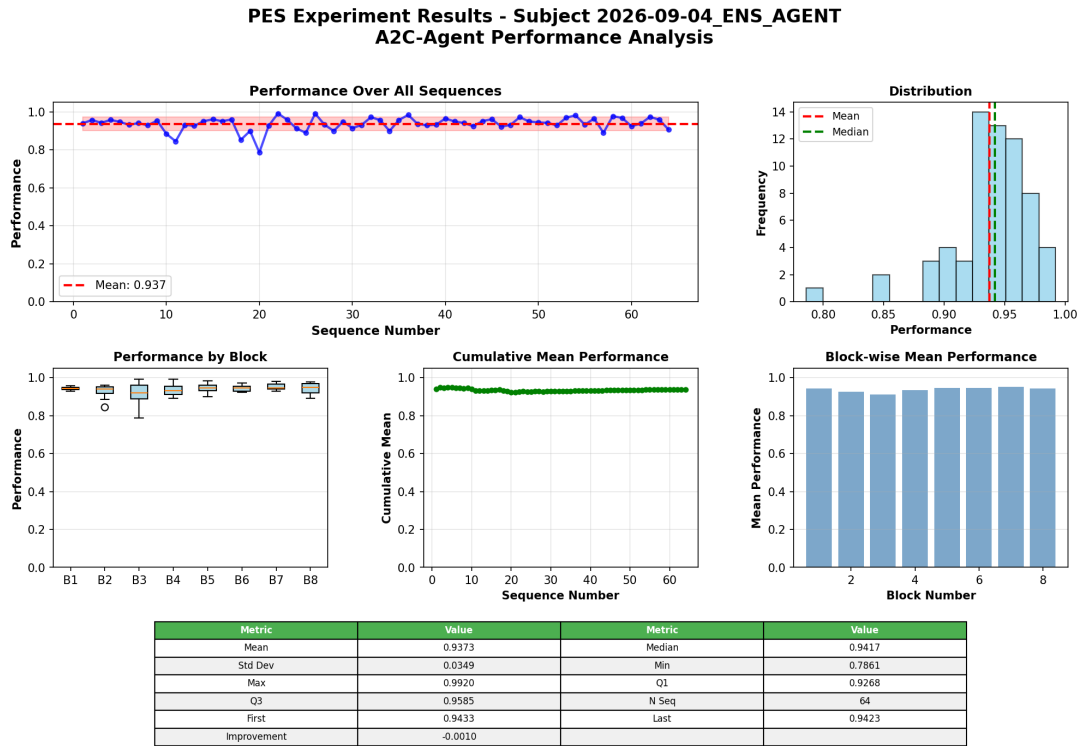


Figura 5.12: Resultados por secuencia del ensemble ponderado (`pes_ens`) en la referencia.

6. Discusión

Los resultados respaldan la hipótesis H1, y la respuesta a la segunda pregunta es afirmativa para una sola de las seis reglas, el ensamble ponderado. Este capítulo interpreta ambos resultados.

6.1. Interpretación de la ventaja del Transformer

Los resultados no identifican la causa de la ventaja del Transformer, pero descartan que se deba sólo a la ventana de estados. El DQN recurrente recibe la misma ventana de seis estados y en generalización queda por debajo del Transformer (0,889 frente a 0,930) y también de DQN, que sólo ve el estado actual (0,899). Esto es compatible con la estructura del entorno: como la asignación de cada ciudad se fija al llegar, la historia no cambia qué acción conviene y sólo aporta información para estimar el retorno (Sección 4.2.2). La ventaja del Transformer sobre DQN existe, pero es de tamaño medio ($d = 0,51$).

Hausknecht y Stone [13] encontraron que la recurrencia ayuda cuando el estado observado omite información necesaria para decidir. En mPES, lo que la ventana puede recuperar, la severidad de las ciudades ya atendidas, afecta a la recompensa pero no a la decisión, lo que es coherente con que el DQN recurrente no mejore a DQN en generalización. Parisotto *et al.* [28] encontraron que el Transformer estándar es difícil de entrenar en RL. El de esta tesis usa una de sus dos modificaciones, la normalización previa a cada subcapa, sin compuertas, y es pequeño: dos bloques sobre una ventana de seis estados.

Tampoco puede atribuirse la ventaja a una búsqueda de hiperparámetros más intensa: el Transformer tuvo 10 ensayos, frente a 47 de DQN, y su arquitectura no es la del mejor ensayo sino la fija de su paquete (Sección 4.4.2). Eso no descarta que la combinación de esa arquitectura con los hiperparámetros de entrenamiento elegidos le resulte especialmente favorable.

En línea con Packer *et al.* [22], que distinguen entre interpolación y extrapolación, cada familia falla al extrapolar un factor distinto. Los modelos tabulares fallan al extrapolar la severidad: la recortan a 9, una fila de la tabla que el entrenamiento nunca visitó. A2C recibe la misma severidad recortada y, sin embargo, mantiene su desempeño: una red extiende

a $S = 9$ lo aprendido para severidades vecinas, mientras que una tabla no generaliza entre filas. Las otras redes reciben $S/9$ sin recortar y, salvo el DQN recurrente, también mantienen su desempeño en ese escenario. Su punto débil es extrapolar la longitud: las secuencias más largas que las de entrenamiento obligan a administrar el presupuesto durante más pasos de los que la política aprendió a anticipar. En ese escenario el mejor modelo es el DQN recurrente (0,89 frente a 0,86 del Transformer), el único caso fuera de rango en que el Transformer no queda primero.

6.2. Efecto de la regla de combinación

Los seis ensambles combinan los mismos modelos entrenados y todos ponderan a cada miembro por su confianza $c_k = 1 - H_{\text{norm}}(p_k)$. Sólo cambia la regla de combinación, y la Tabla 5.4 permite ver quién decide en cada caso:

- El **ensamble ponderado** le da al Transformer alrededor del 82 % del peso base, pero no es el Transformer el que decide. Con $\tau = 15$ las distribuciones de los miembros son casi planas, y el factor que penaliza no asignar nada y el *prior* de severidad pesan tanto como ellas. La acción final difiere de la del Transformer en el 61 % de las decisiones de la referencia y en el 48 % de las de generalización, y el *prior* cambia la acción votada en el 41 % y el 33 %, respectivamente; la cota de seguridad casi nunca actúa. Su ventaja proviene, por lo tanto, de combinar una preferencia débil de los modelos con una regla fija que asigna aproximadamente tantos recursos como la severidad de la ciudad.
- La **compuerta del Transformer** sigue al Transformer en el 97 % de las decisiones de la referencia y en el 95 % de las de generalización, y cambia su acción en menos del 3 %. Por eso sus resultados son casi idénticos a los del Transformer (0,931 frente a 0,930 en generalización).
- El **voto suave** y el **voto por acción** usan pesos en los que DQN pesa más que el Transformer. En la extrapolación de severidad quedan por debajo de ambos y sólo superan al DQN recurrente: la ponderación por confianza no alcanzó para que el miembro más acertado dominara la decisión.
- El **consenso** obtiene la referencia más baja. Agregarle el *prior* de severidad mejora

tanto su referencia como su peor escenario, aunque el *prior* sólo cambia el 8 % de sus decisiones en la referencia.

El único ensamble que superó al Transformer no es el que más lo sigue. La compuerta lo sigue casi siempre y lo iguala, mientras que el ensamble ponderado se aparta de su acción en la mayoría de las decisiones de la referencia y en casi la mitad de las de generalización. Los datos son compatibles con que la mejora provenga del *prior* de severidad y de la temperatura alta, que suaviza las preferencias de los modelos, más que de la ponderación por confianza. Con un solo ensamble ganador no puede separarse el aporte de cada componente. Usada como único criterio para promediar, la confianza no bastó.

7. Conclusiones

Esta tesis comparó seis implementaciones de aprendizaje por refuerzo y seis formas de combinarlas en un ensamble, evaluadas sobre las mismas secuencias en la condición de entrenamiento y en 21 escenarios de generalización.

7.1. Respuesta a las preguntas

¿Cuál es el mejor modelo individual? El Transformer, lo que respalda la hipótesis H1. En los escenarios de generalización obtiene 0,930, por encima de todos los demás modelos, y en la referencia reduce en alrededor de un 36 % la distancia al óptimo que deja Q-Learning. Su ventaja es de tamaño medio frente a DQN y al DQN recurrente. Conserva el desempeño con severidades fuera de rango, donde fallan los métodos tabulares, y lo pierde en parte con secuencias más largas que las de entrenamiento, el punto débil de todas las redes. No es, en cambio, el que menos pierde respecto de su propia referencia: su peor escenario cae 0,067, frente a 0,053 de DQN.

¿Algún ensamble lo supera? Sí, el ensamble ponderado, con 0,939 en generalización y un peor escenario más alto que el del Transformer. Es el único de los seis: la compuerta del Transformer lo iguala y los otros cuatro quedan por debajo. Aunque le da al Transformer la mayor parte del peso base, en la referencia su acción final difiere de la del Transformer en el 61 % de las decisiones, sobre todo por el *prior* de severidad. Su ventaja no se debe, por lo tanto, a ponderar a los modelos por su confianza.

7.2. Limitaciones del estudio

En primer lugar, la referencia usa un único conjunto fijo de 64 secuencias, y cada escenario de generalización se generó con una única semilla (42). Eso garantiza que todos los modelos enfrenten las mismas secuencias, pero hace que cada media dependa de una única muestra de 64 secuencias por escenario. En los escenarios de generalización no era necesario fijar la semilla: con semillas distintas se habría podido estimar la variabilidad de cada media y dar intervalos de confianza para el ordenamiento de los modelos.

En segundo lugar, los hiperparámetros de los modelos y de los ensambles se ajustaron con las mismas 64 secuencias con que después se evaluó la referencia. Además, el entorno de entrenamiento sortea las secuencias a partir de los mismos archivos de severidades iniciales y longitudes. Por eso los valores de la referencia pueden ser algo optimistas. Los escenarios de generalización están menos expuestos a ese sesgo, porque sus secuencias provienen de otras distribuciones.

En tercer lugar, el entrenamiento sólo presentó severidades de 2 a 8. En los métodos tabulares, las filas de severidad 0, 1 y 9 quedaron con sus valores iniciales aleatorios, de modo que parte de su caída en los escenarios de severidad refleja esa falta de cobertura del entrenamiento y no sólo la capacidad de generalizar del algoritmo.

En cuarto lugar, los modelos evaluados no siempre son la mejor configuración que encontró cada búsqueda. El DQN recurrente y el Transformer finales conservan la arquitectura fija de su paquete y no la del mejor ensayo; el DQN recurrente final queda 0,027 por debajo de ese ensayo. En cuatro de los ensambles, los parámetros provienen de estudios hechos con otra configuración de miembros. Las comparaciones se refieren, por lo tanto, a los modelos finales evaluados.

En quinto lugar, la confianza con que los ensambles ponderan a cada modelo se calcula sobre una distribución derivada de los valores Q y no sobre probabilidades aprendidas, y no se comprobó que sea mayor en las decisiones acertadas.

Por último, los resultados no identifican la causa de la ventaja del Transformer, y todas las conclusiones se limitan a este entorno y a los modelos entrenados.

7.3. Trabajo futuro

- **Evaluación con semillas variables.** Generar cada escenario con semillas independientes para dar intervalos de confianza al ordenamiento de la Tabla 5.1.
- **Validación de la medida de confianza.** Relacionar $1 - H_{\text{norm}}$ con el desempeño por secuencia mediante curvas de calibración, y extender el cálculo a los demás modelos con un procedimiento de calibración común.
- **Pesos dependientes del contexto en el ensamble.** Los pesos base w_k son constantes; una extensión es aprender pesos $w_k(s)$ que dependan del estado.

7.4. Conclusión general

En el escenario controlado de mPES, el Transformer causal es el modelo individual que mejor decide, tanto en las condiciones de entrenamiento como cuando éstas cambian, aunque no es el que menos pierde respecto de su propia referencia. Combinar modelos lo mejoró en un solo caso: el ensamble que suaviza las preferencias de los modelos y las mezcla con un *prior* de severidad, que cambia buena parte de sus decisiones.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic experiment scenario. <https://github.com/BCI-NE/PES>, 2023. Accedido: septiembre 2026.
- [2] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021.
- [3] Andrea Parisi, Samuel P. C. Brand, Joe Hilton, Rabia Aziza, Matt J. Keeling, and D. James Nokes. Spatially resolved simulations of the spread of COVID-19 in three European countries. *PLOS Computational Biology*, 17(7):e1009090, 2021. doi: 10.1371/journal.pcbi.1009090.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [5] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [6] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [7] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, 1999.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [9] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- [10] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.
- [11] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [13] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [14] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [15] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

- [16] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [18] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [19] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [20] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [21] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.
- [22] Charles Packer, Katelyn Gao, Jernej Kos, Philipp Krähenbühl, Vladlen Koltun, and Dawn Song. Assessing generalization in deep reinforcement learning. *arXiv preprint arXiv:1810.12282*, 2018. URL <https://arxiv.org/abs/1810.12282>.
- [23] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [24] Bernard L. Welch. The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [25] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [26] Tasos Papastylianou, Rodrigo Ramele, Luca Citi, Caterina Cinel, and Riccardo Poli. PES – pandemic emergency scenario. OpenNeuro, conjunto de datos ds004477, versión 1.0.2, 2023. URL <https://doi.org/10.18112/openneuro.ds004477.v1.0.2>.
- [27] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision Transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [28] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.

- [29] Marco A. Wiering and Hado van Hasselt. Ensemble algorithms in reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(4):930–936, 2008. doi: 10.1109/TSMCB.2008.920231.
- [30] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.
- [31] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [32] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [33] Maximiliano Vega. mPES: Multiple pandemic experiment suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: septiembre 2026.
- [34] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, K. G. Arjun, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.
- [35] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [36] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9, pages 249–256, 2010.

Apéndice

A. Comandos de reproducción

La Tabla A.1 resume los comandos que regeneran los resultados de esta tesis. El entorno virtual (`win_mpes_env`, en Windows) se activa desde la raíz del repositorio `mPES`, y todos los comandos se ejecutan desde el directorio `h1/`, con Python 3.12, las dependencias fijadas en `utils/config/requirements.txt` y las variables `VIRTUAL_ENV`, `PYTHONIOENCODING=utf-8` y `TF_ENABLE_ONEDNN_OPTS=0`. El entrenamiento final lee los hiperparámetros del archivo `inputs/best_params.json` de cada paquete. En DQN, DQN recurrente y Transformer la arquitectura sale de `config/CONFIG.py`, cuyos valores por defecto (incluido el número de episodios) reproducen los modelos desplegados.

Tabla A.1: Comandos de optimización bayesiana, entrenamiento final y evaluación de cada paquete, ejecutados desde `h1/`. `<pkg>` es cualquiera de los cinco ensambles optimizables.

Paquete	Optimización bayesiana	Entrenamiento final	Evaluación
<code>pes_base</code>	—	<code>python -m tabular.pes_base.ext.train_rl</code>	<code>python -m tabular.pes_base</code>
<code>pes_q1</code>	<code>python -m tabular.pes_q1.ext.optimize_rl</code>	<code>python -m tabular.pes_q1.ext.train_rl</code>	<code>python -m tabular.pes_q1</code>
<code>pes_dql</code>	<code>python -m tabular.pes_dql.ext.optimize_rl</code>	<code>python -m tabular.pes_dql.ext.train_rl</code>	<code>python -m tabular.pes_dql</code>
<code>pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>	<code>python -m ml.pes_dqn.ext.train_dqn</code>	<code>python -m ml.pes_dqn</code>
<code>pes_rdn</code>	<code>python -m ml.pes_rdn.ext.optimize_rdn</code>	<code>python -m ml.pes_rdn.ext.train_rdn</code>	<code>python -m ml.pes_rdn</code>
<code>pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>	<code>python -m ml.pes_a2c.ext.train_a2c</code>	<code>python -m ml.pes_a2c</code>
<code>pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>	<code>python -m ml.pes_trf.ext.train_transformer</code>	<code>python -m ml.pes_trf</code>
<code>pes_ens</code>	—	—	<code>python -m ens.pes_ens</code>
<code><pkg></code>	<code>python -m ens.<pkg>.ext.optimize_ens</code>	—	<code>python -m ens.<pkg></code>

B. Ejecución de la evaluación

La evaluación en los 22 escenarios se ejecuta con tres comandos desde `h1/`; el primero es reanudable:

```
python -m general.scripts.benchmark run --suite both
python -m general.scripts.analysis
python -m general.scripts.figures
```

El primero evalúa cada par (modelo, escenario) de los dos grupos de modelos (`individual` y `ensemble`), reutilizando los modelos entrenados y sustituyendo temporalmente sus archivos de entrada por los del escenario. El segundo calcula las matrices `modelo × escenario` de media, desviación estándar, mínimo, máximo, degradación, p de Welch, d de Cohen y KL de acciones. El tercero genera las

figuras y las comparaciones entre pares de modelos. Los resultados quedan en `general/results/<grupo>/: cells/` (desempeño por secuencia y distribución de acciones de cada par modelo–escenario), `matrices/`, `comparison_metrics.json`, `figures/` y un resumen `report.md`. Las figuras del agente aleatorio y de la confianza del Transformer se generan con `python -m general.scripts.random_baseline` y `python -m general.scripts.agent_internals`.

Las frecuencias de la Tabla 5.4 se obtienen, desde la raíz del repositorio, con `python writings/auxiliar/scripts/ensemble_decisions.py`. El script repite la evaluación del ensamble ponderado, del consenso con *prior* y de la compuerta del Transformer sobre los mismos archivos de cada escenario. En cada decisión con recursos disponibles registra la acción antes y después del *prior* y de la cota, y la acción que prefiere el Transformer. También comprueba que las medias coincidan con las del benchmark.

C. Mapas estadísticos complementarios

Las Figuras C.1 y C.2 indican, para cada modelo individual y cada escenario, si el cambio de desempeño respecto de la referencia es estadísticamente claro (prueba de Welch). También muestran cuánto cambia la distribución de acciones, es decir, la política. La Figura C.3 compara todos los pares de modelos individuales. Las Figuras C.4 y C.5 son las equivalentes para los ensambles.

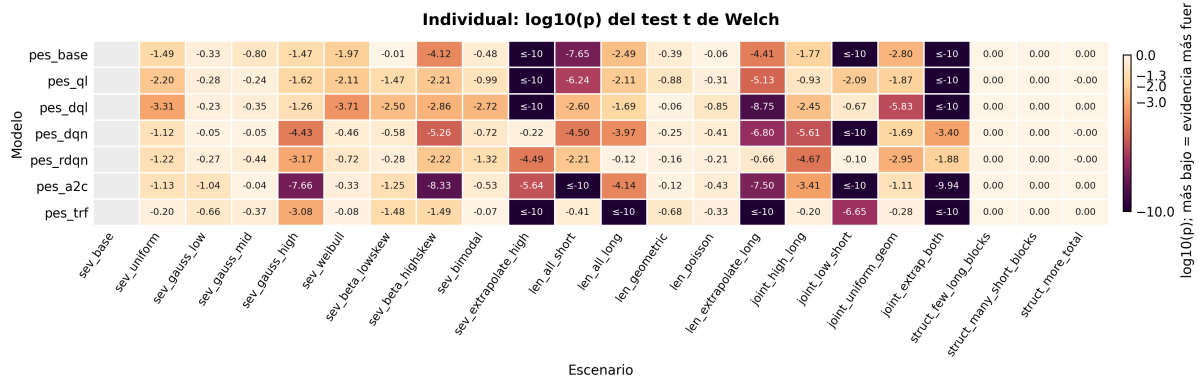


Figura C.1: $\log_{10} p$ de la prueba de Welch entre el desempeño de cada escenario y el de la referencia del mismo modelo; valores más negativos indican un cambio más claro.

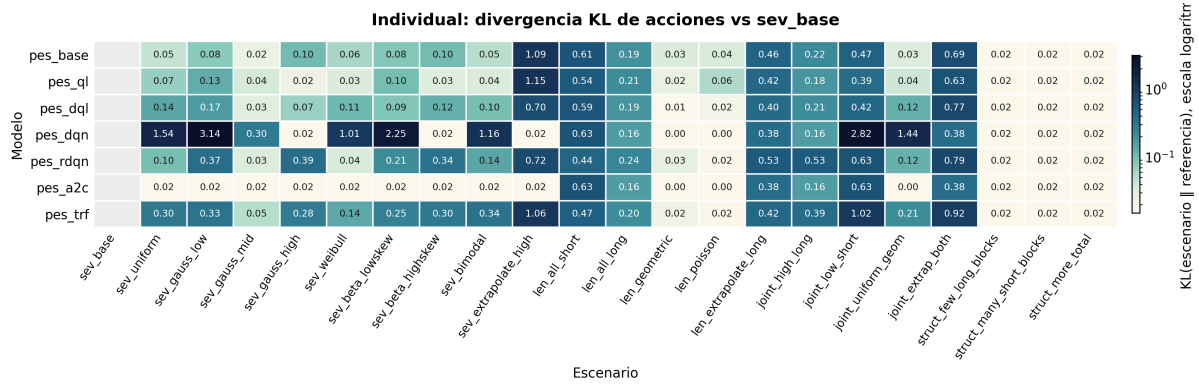


Figura C.2: Divergencia KL entre la distribución de acciones de cada escenario y la de la referencia del mismo modelo: mide cuánto cambia la política, no el desempeño.

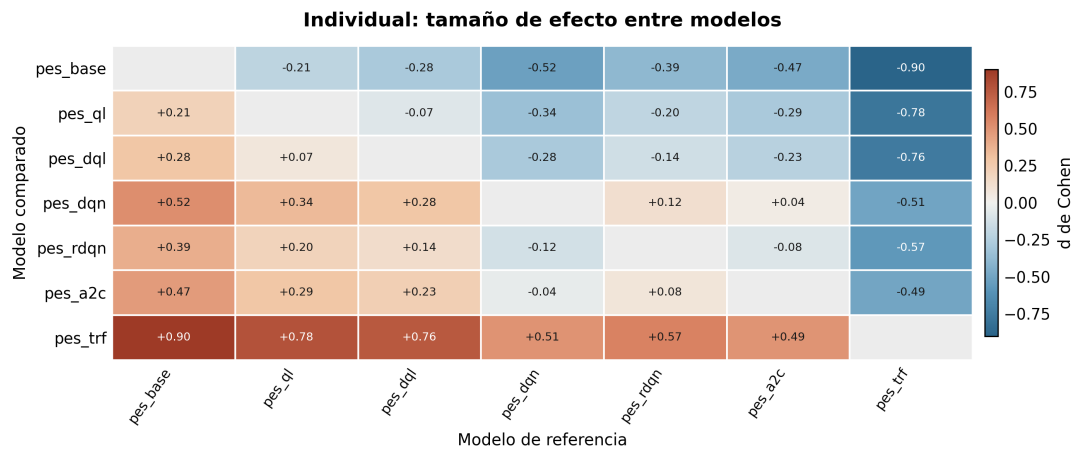


Figura C.3: d de Cohen entre todos los pares de modelos individuales sobre los 21 escenarios de generalización; un valor positivo indica que el modelo de la fila supera al de la columna.

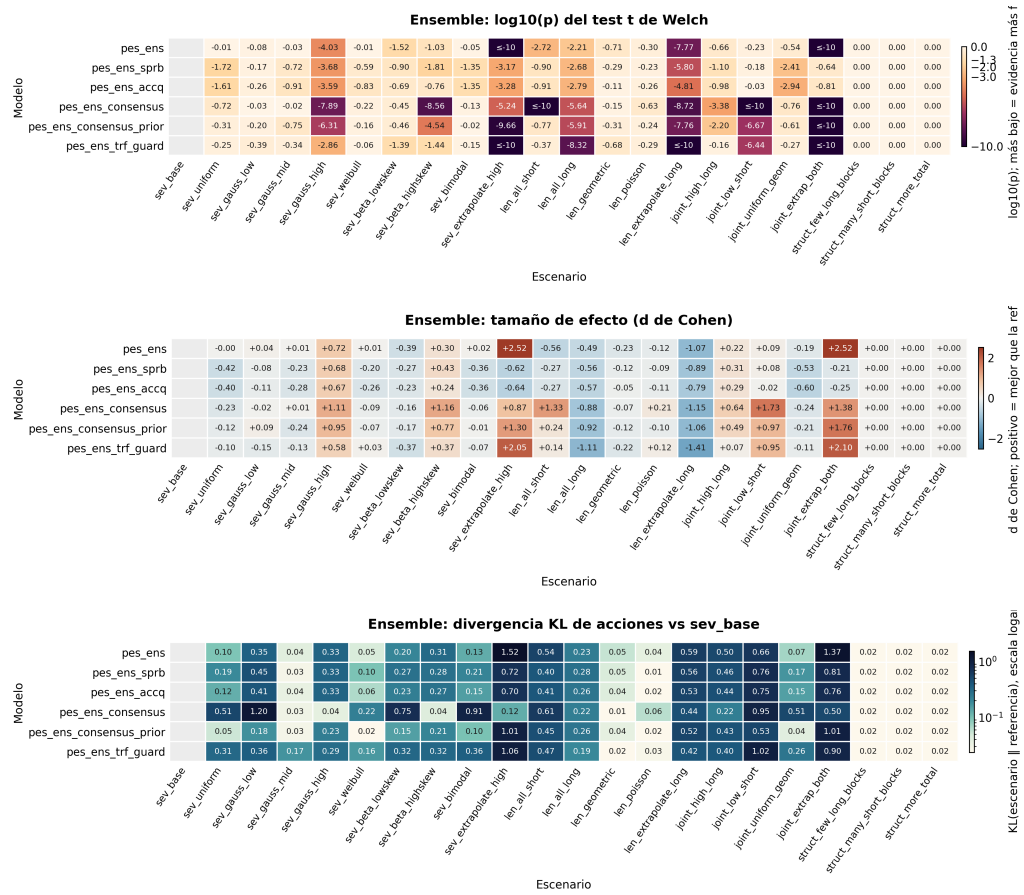


Figura C.4: Ensamblas: $\log_{10} p$ de Welch (arriba), d de Cohen (centro) y divergencia KL de acciones (abajo) de cada escenario frente a la referencia del mismo ensemble.

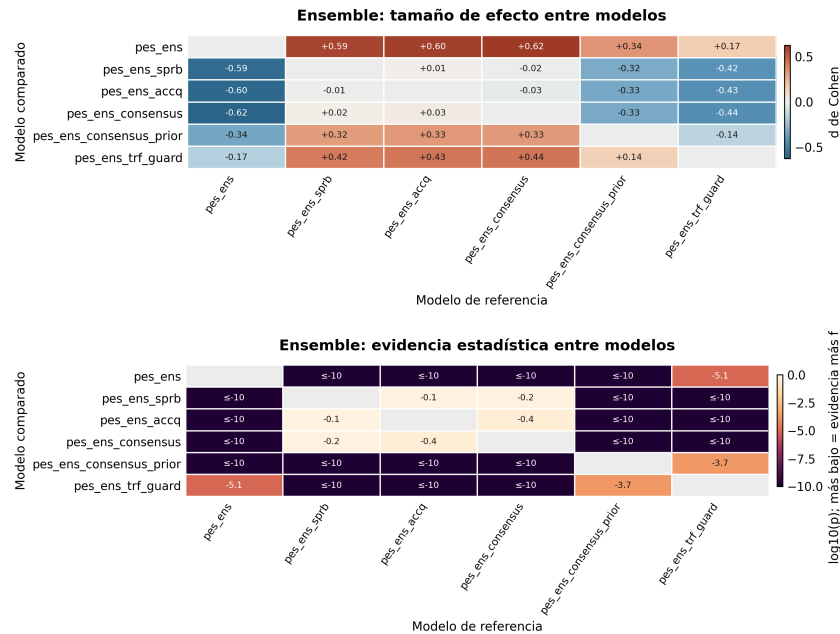


Figura C.5: d de Cohen (arriba) y $\log_{10} p$ de Welch (abajo) entre todos los pares de ensambles sobre los 21 escenarios de generalización.

D. Confianza registrada del Transformer

La Figura D.1 muestra la confianza del Transformer en cada una de las 258 decisiones con recursos disponibles, registrada al evaluarlo sobre las 64 secuencias de la referencia. En este registro la confianza se calcula de otra forma que en los ensambles (Sección 4.5), por lo que sus valores no son comparables con los umbrales de la Tabla 4.6. Los valores Q de las acciones no factibles se reemplazan por un valor muy negativo, y el vector se desplaza a valores no negativos y se normaliza. La confianza es $1 - H_{\text{norm}}$ de esa distribución. Cuando hay acciones no factibles, el desplazamiento acerca la distribución a una uniforme sobre las acciones disponibles, de modo que la confianza refleja sobre todo cuántas acciones quedan y no cuánto difieren sus valores. En la figura, la mayoría de las decisiones tiene una confianza baja (media 0,116), lo que es coherente con esa limitación.

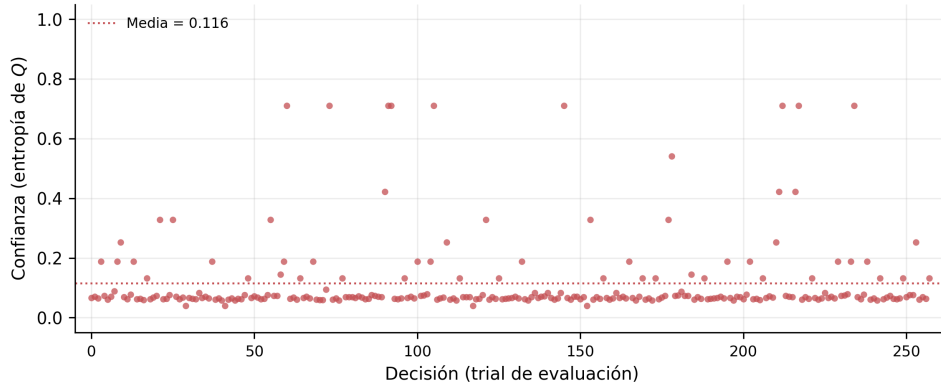


Figura D.1: Confianza $1 - H_{\text{norm}}$ del Transformer en cada una de las 258 decisiones con recursos disponibles; la línea punteada indica la media.